UNIVERSITÄT PASSAU

Software Analysis

Interprocedural and Heap Analysis

Gordon Fraser
Lehrstuhl für Software Engineering II

Outline

1. Limitations of Intraprocedural Analysis
2. Interprocedural Analysis
3. Dataflow Analysis and the Heap

Limitations of Intra-Procedural Analysis

```
int a = 7;
int d = f(a, 2);
int e = a + d;
```

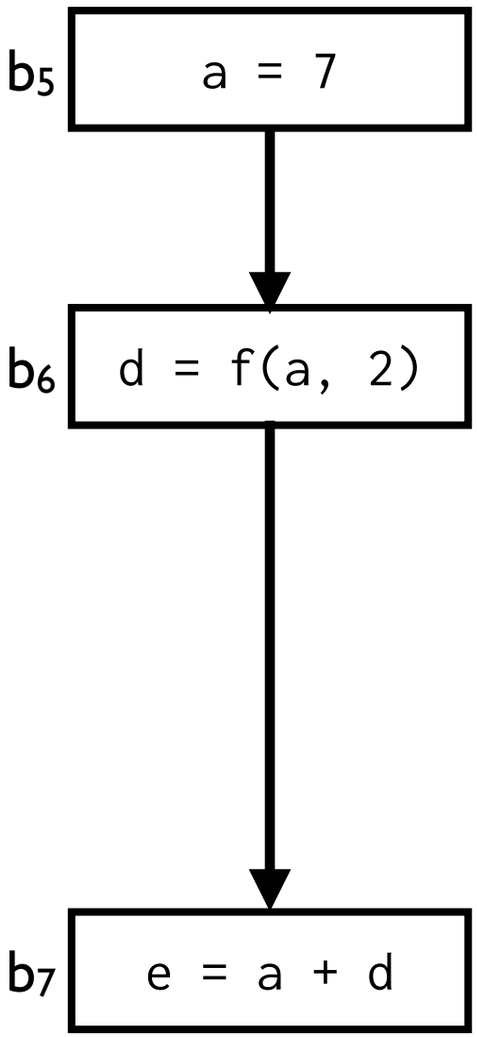
If you do not know anything about methods or functions the analysis must be conservative: it is $\top$ or $\bot$

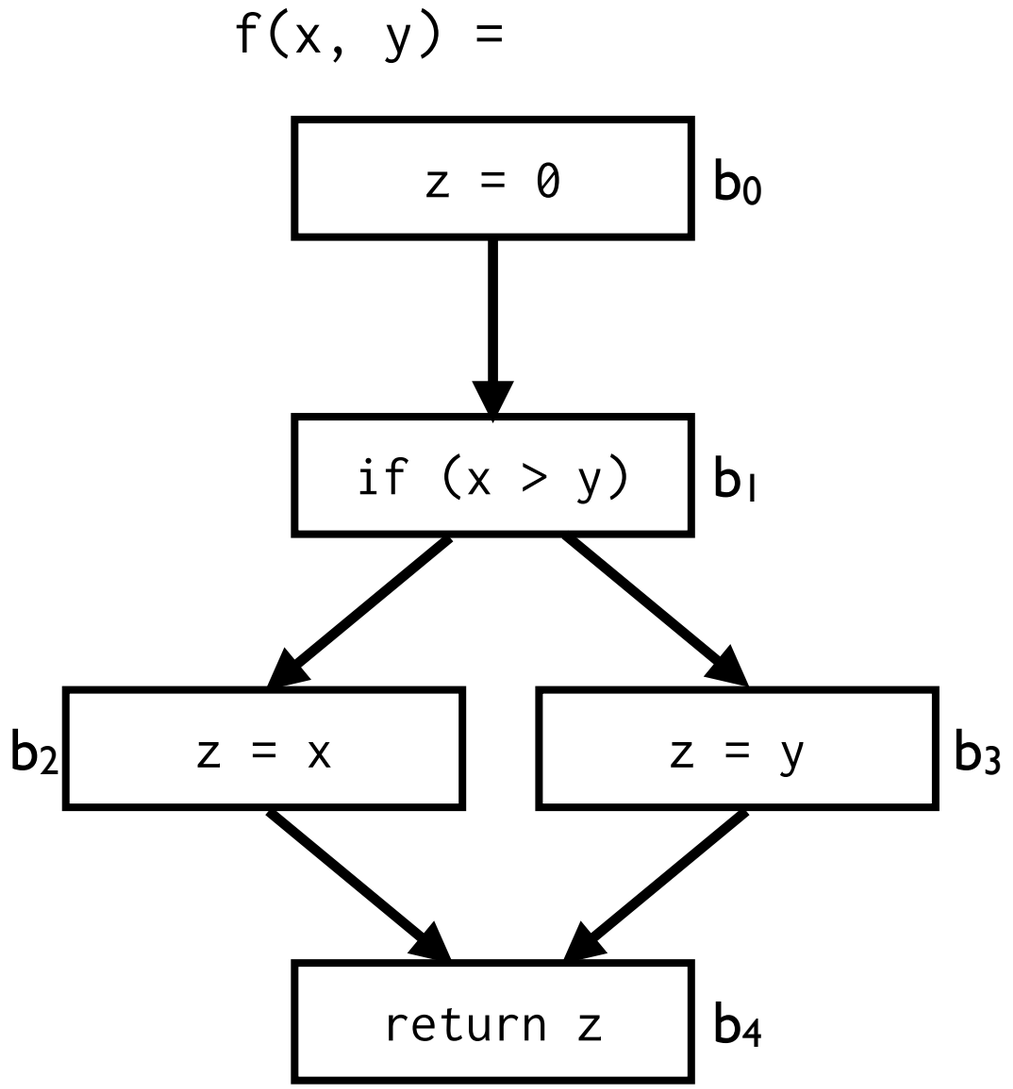
Limitations of Intra-Procedural Analysis

```
int a = 7;
int d = f(a, 2);
int e = a + d;
```

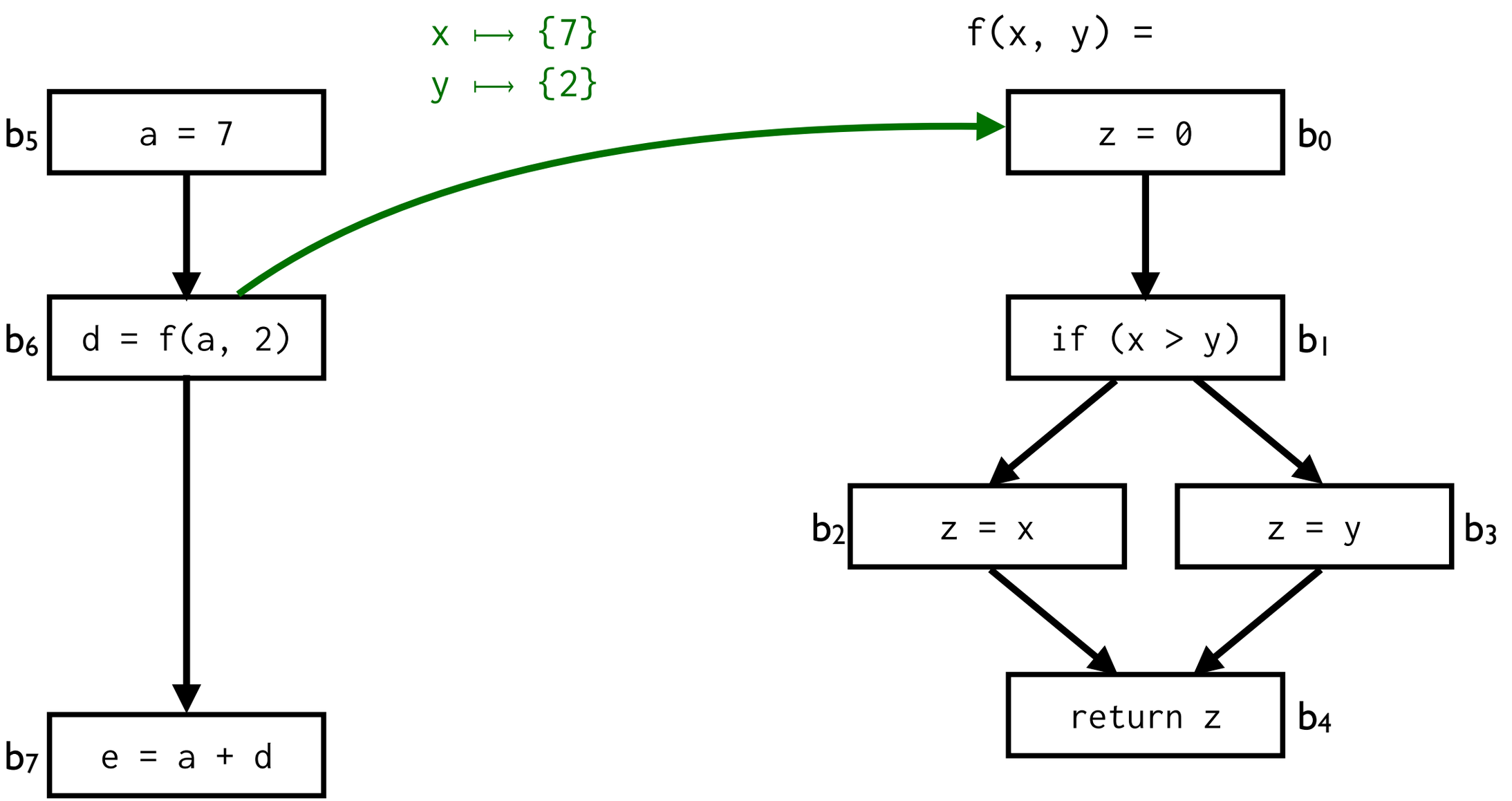
```
int f(int x, int y) {
  int z = 0;
  if (x > y) {
    z = x;
  } else {
    z = y;
  }
  return z;
}
```

Limitations of Intra-Procedural Analysis

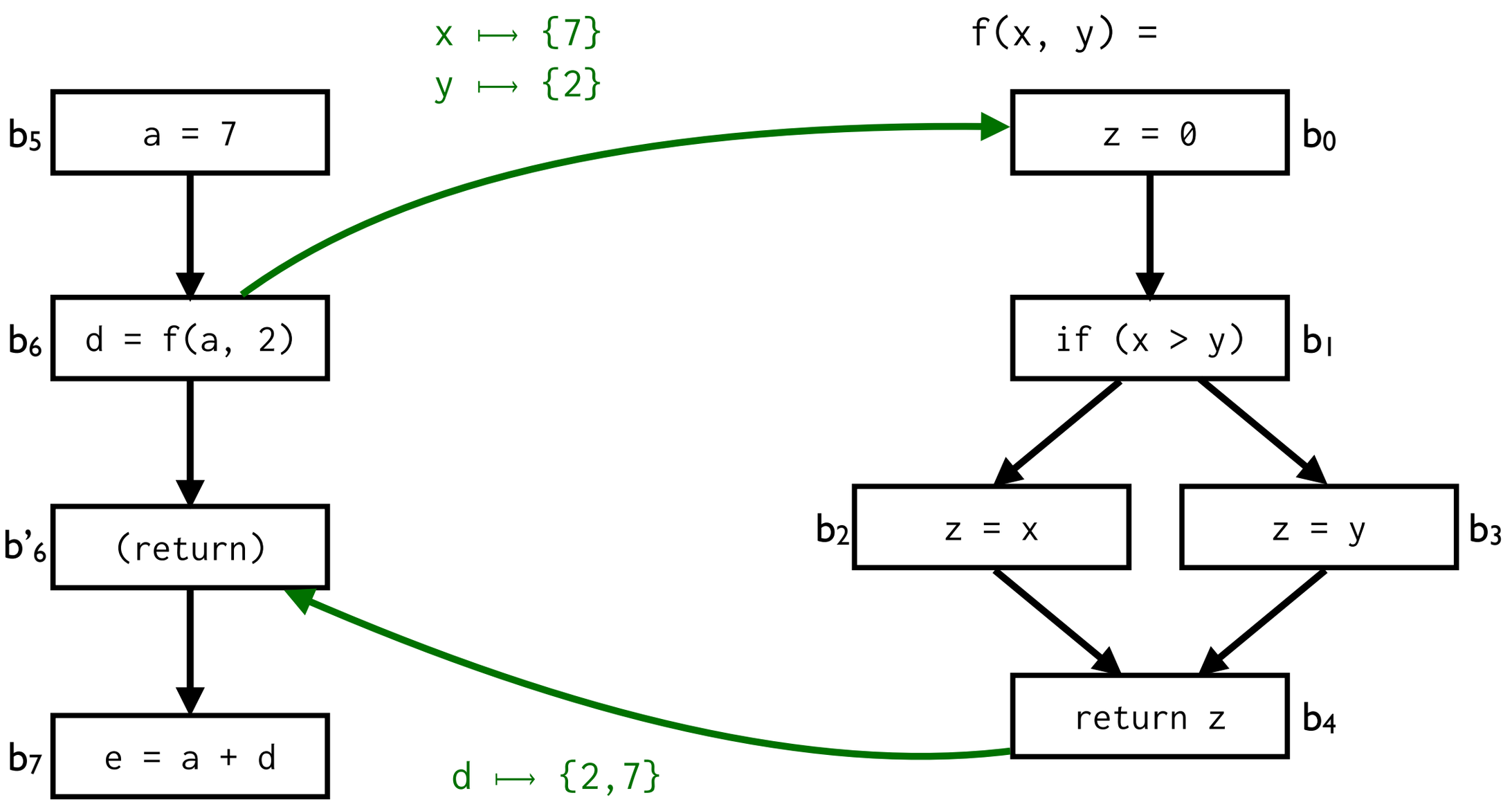




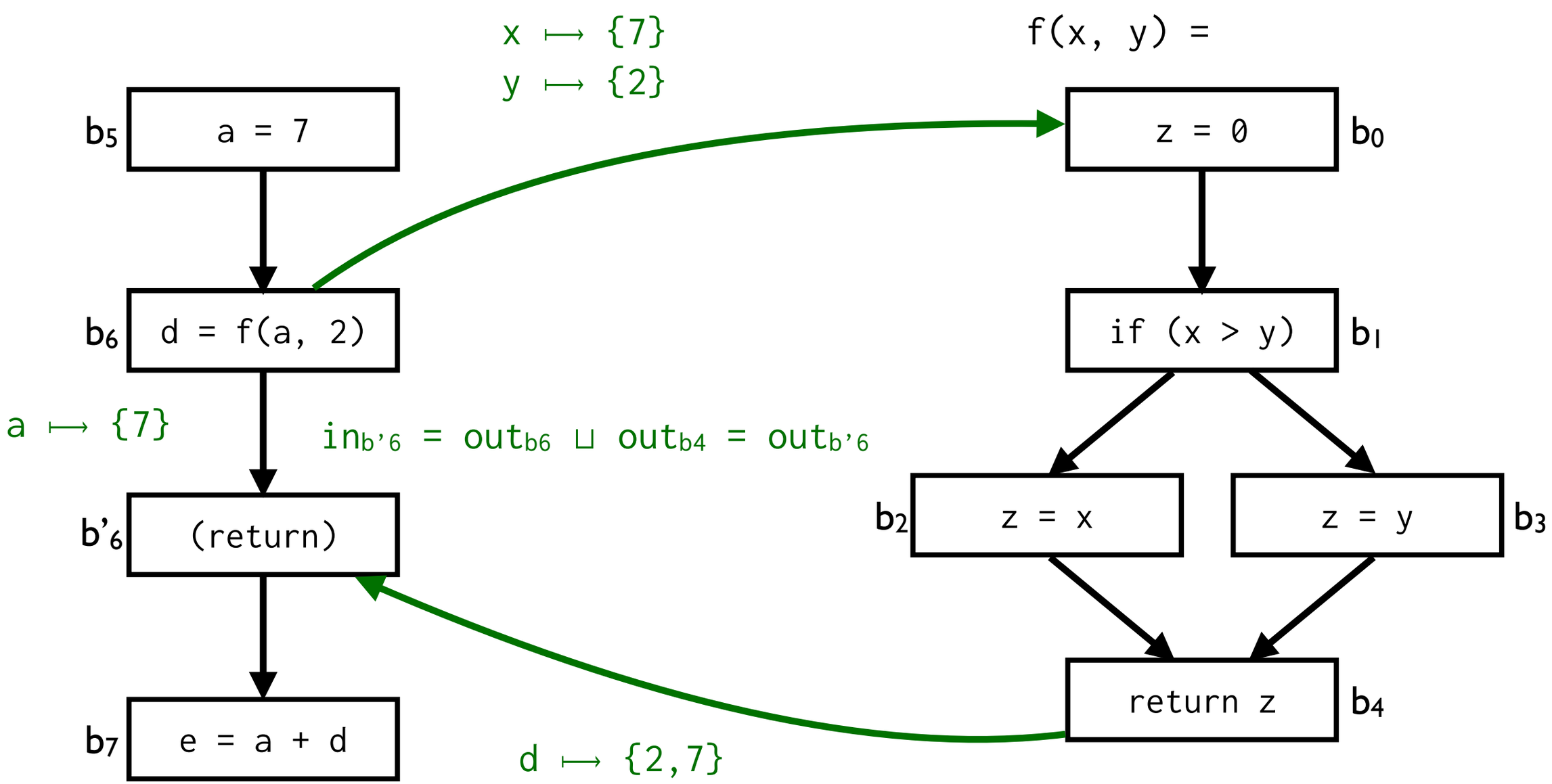
Limitations of Intra-Procedural Analysis



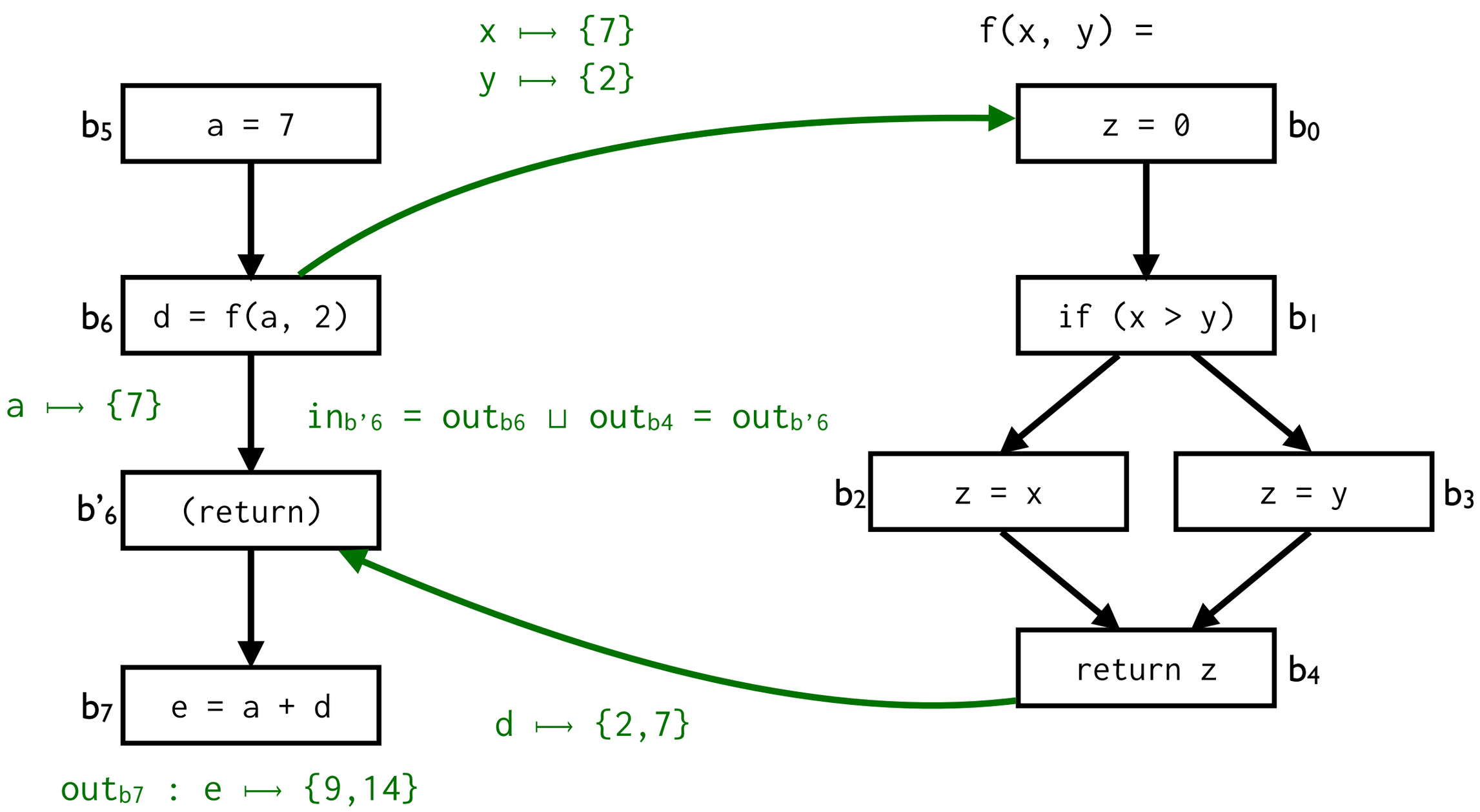
Limitations of Intra-Procedural Analysis



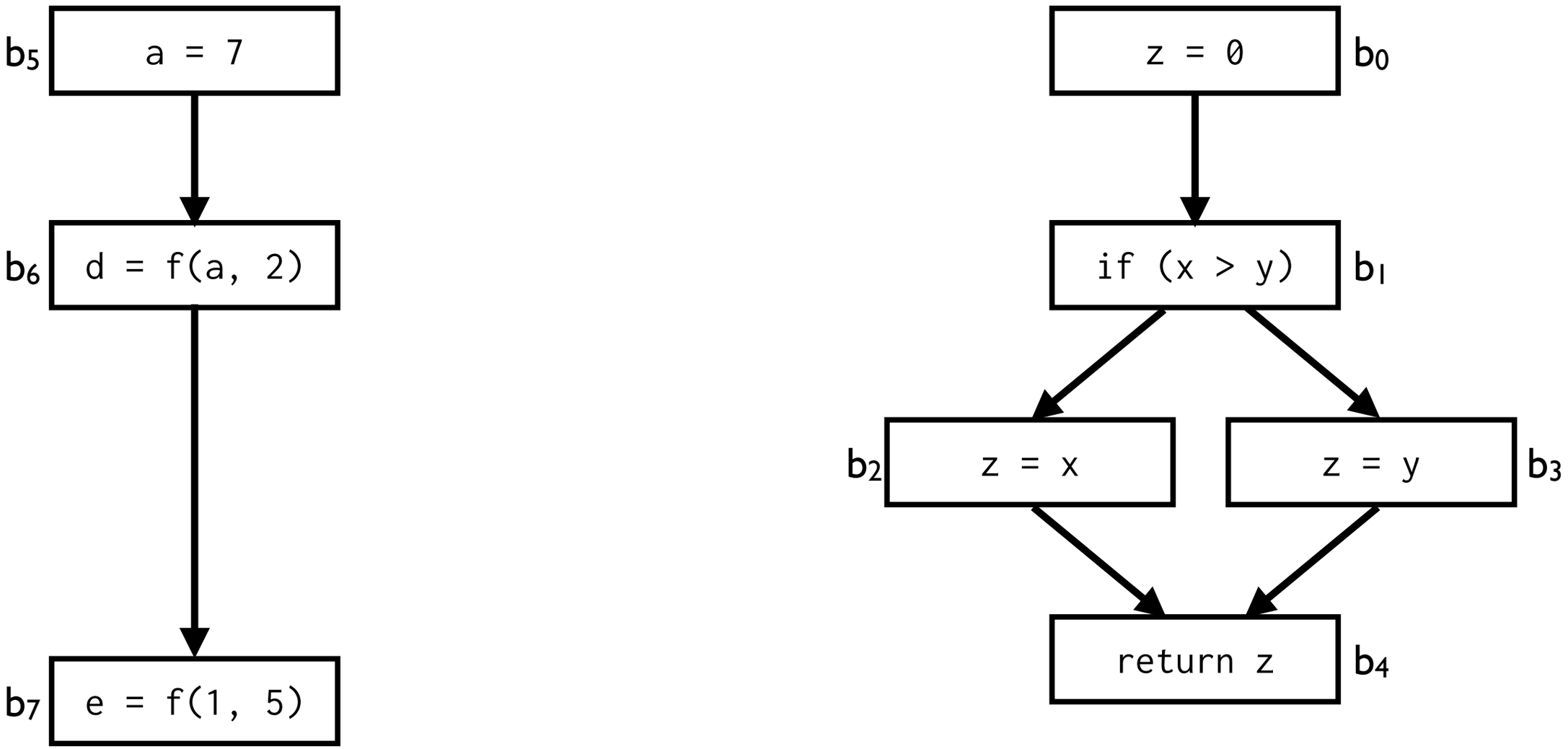
Limitations of Intra-Procedural Analysis



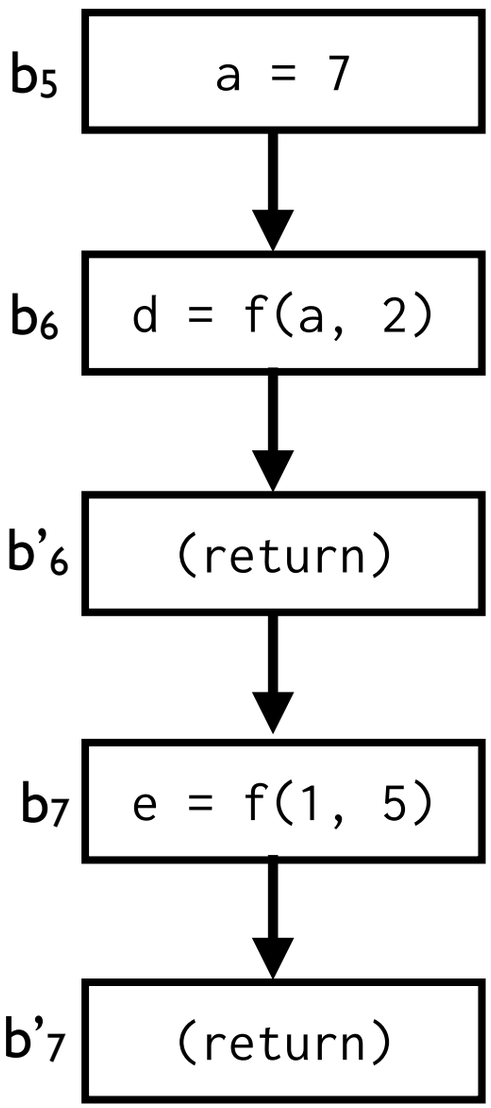
Limitations of Intra-Procedural Analysis

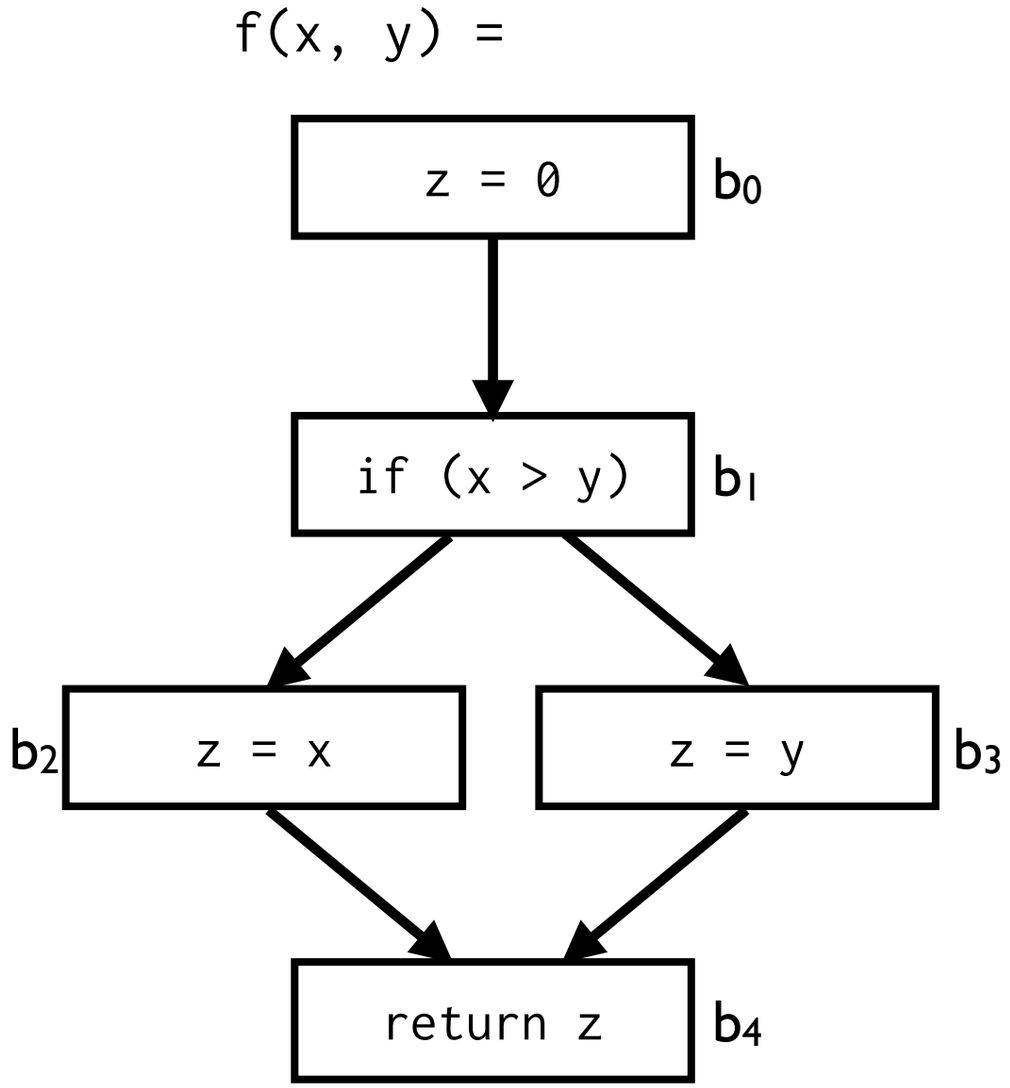


Limitations of Intra-Procedural Analysis

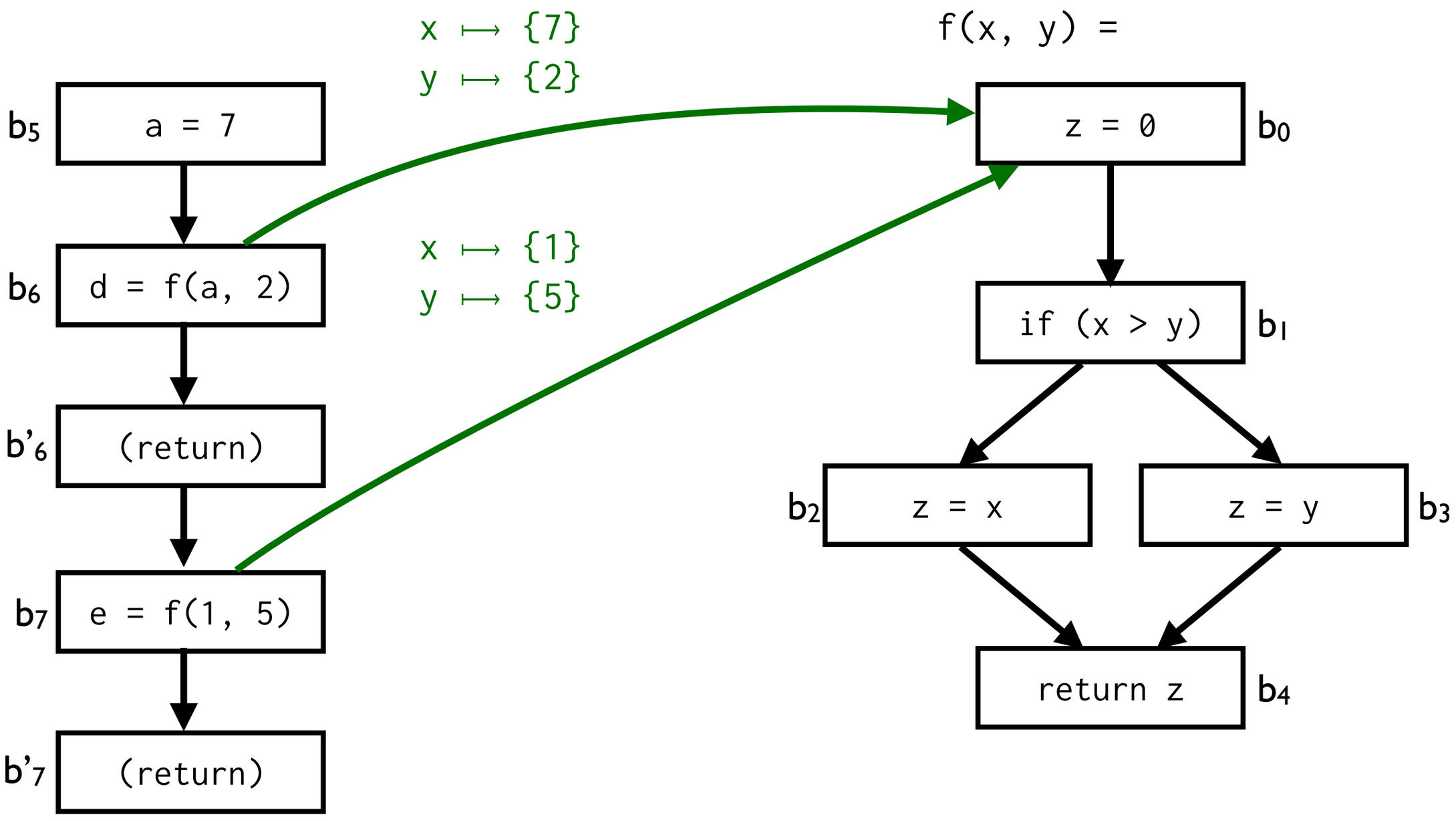


Limitations of Intra-Procedural Analysis

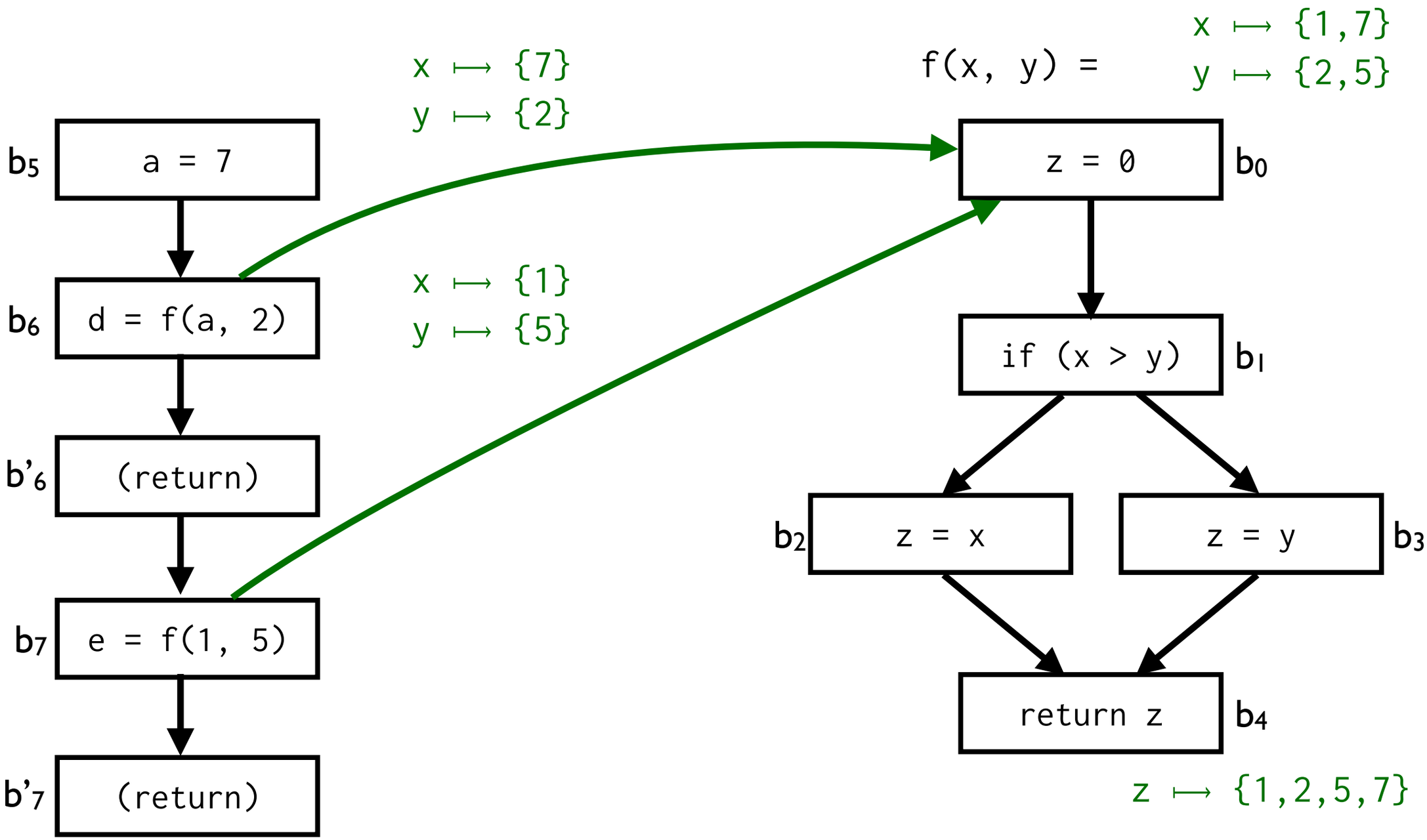




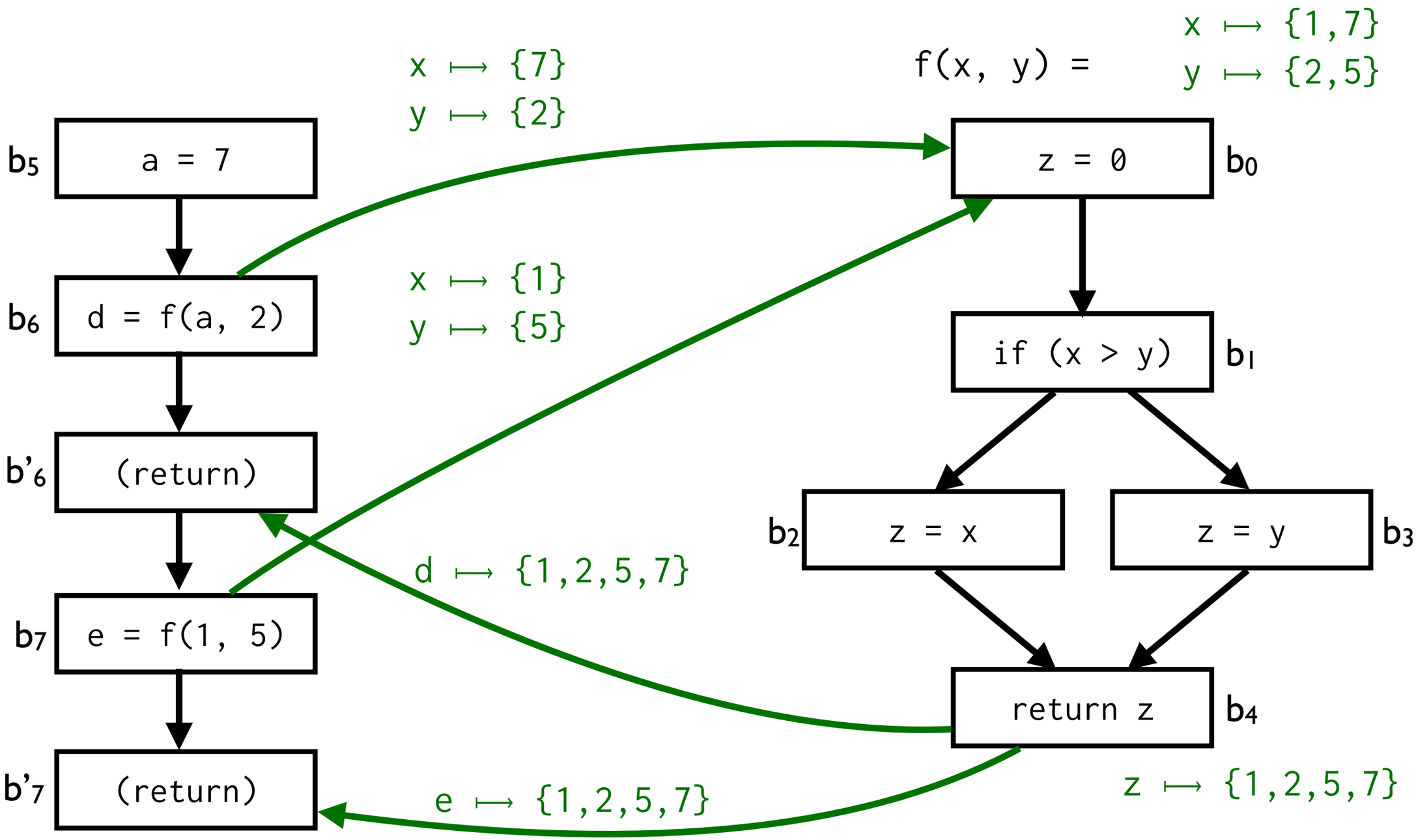
Limitations of Intra-Procedural Analysis



Limitations of Intra-Procedural Analysis



Limitations of Intra-Procedural Analysis



Limitations of Intra-Procedural Analysis

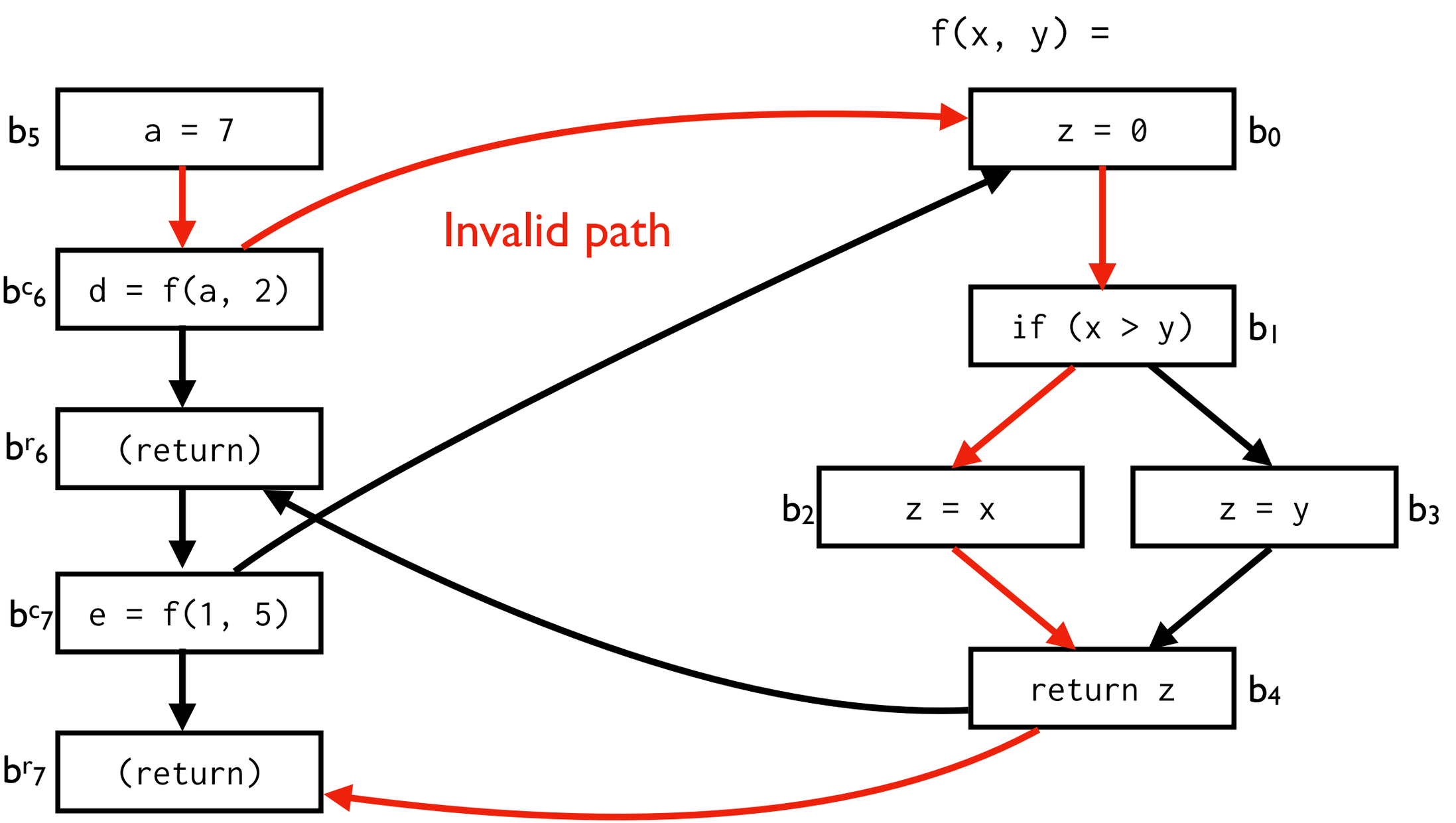
- Split call sites b_x into call (b^c_x) and return (b^r_x) nodes
- Intra-procedural edge:
  - $b^c_x \rightarrow b^c_x$ carries environment/store
- Inter-procedural edge:
  - Caller → subroutine, substitutes parameters (for pass-by-value)
  - Return → Caller, substitutes result (for pass-by-result)

Limitations of Intra-Procedural Analysis

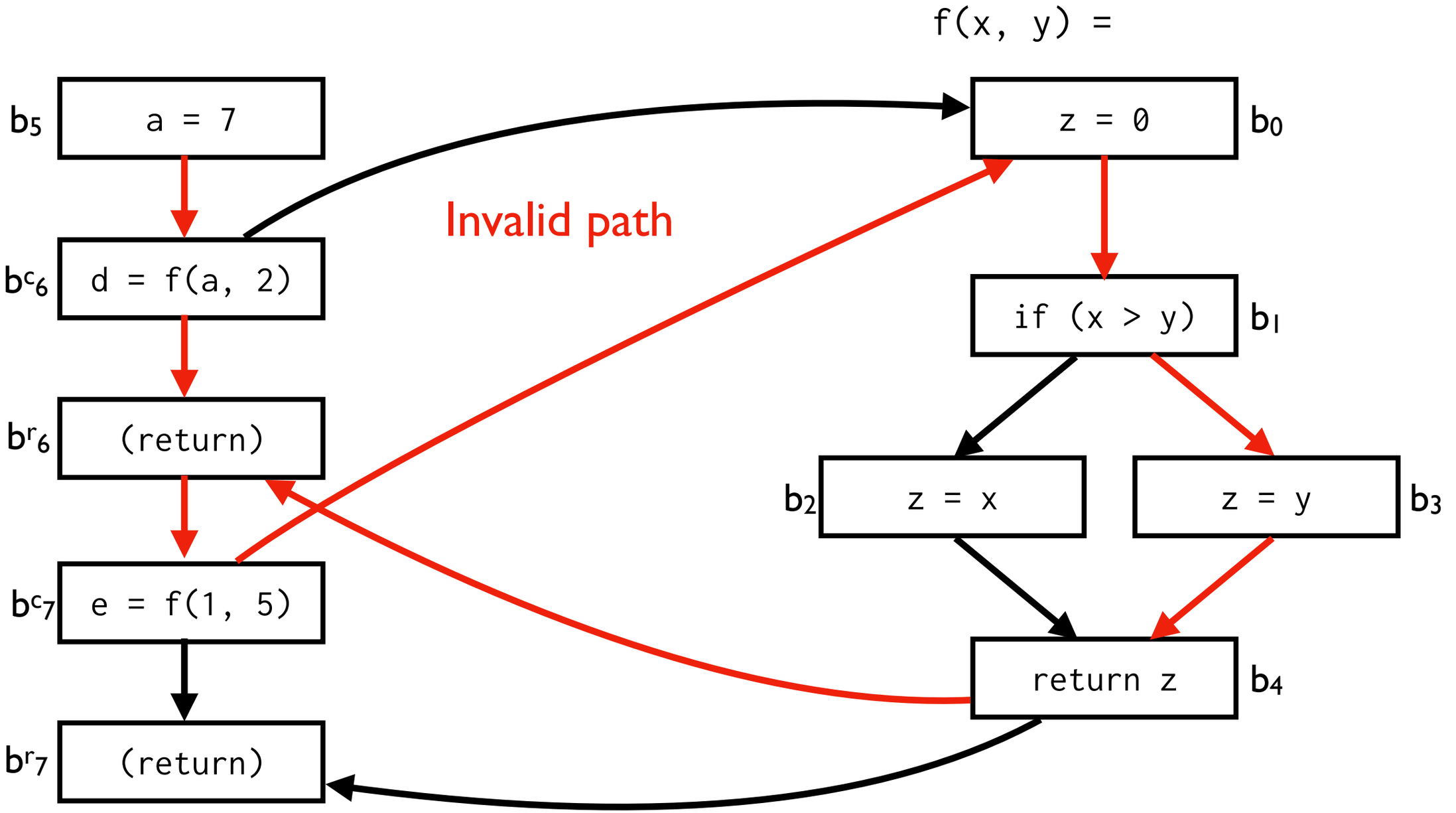
- In this **naive** approach, **all call sites** connected to subroutine entry and subroutine return connected to **all return sites**
- So the analysis is **context-insensitive** (it cannot distinguish between the different call contexts)

Meet over valid paths

Valid Paths

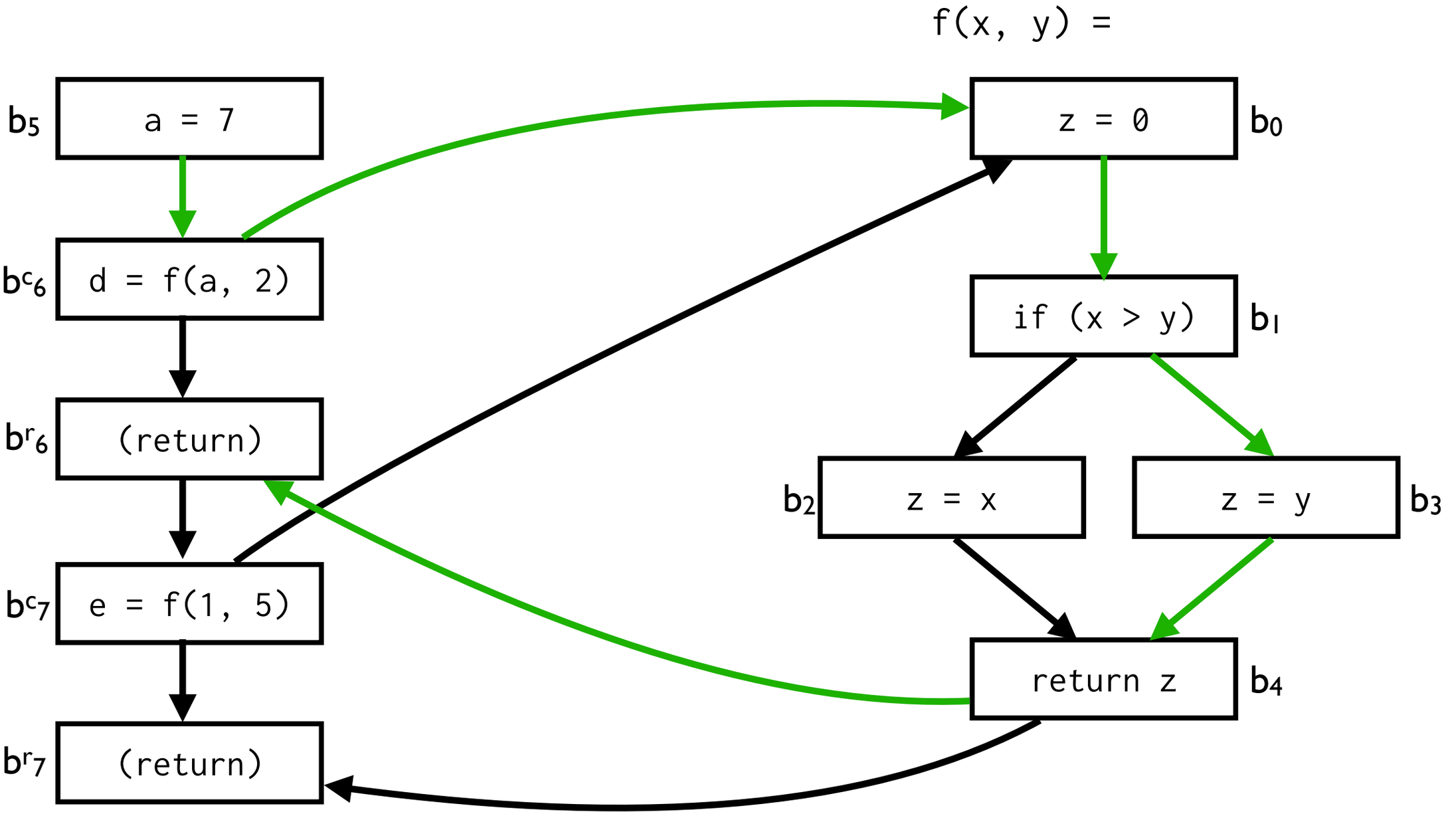

f(x, y) =
b5
a = 7
bc6
d = f(a, 2)
br6
(return)
bc7
e = f(1, 5)
br7
(return)
Invalid path
z = 0
b0
if (x > y)
b1
b2
z = x
z = y
b3
return z
b4

Valid Paths


f(x, y) =
b5
a = 7
z = 0
b0
Invalid path
bc6
d = f(a, 2)
if (x > y)
b1
br6
(return)
b2
z = x
z = y
b3
bc7
e = f(1, 5)
return z
b4
br7
(return)

Valid Paths

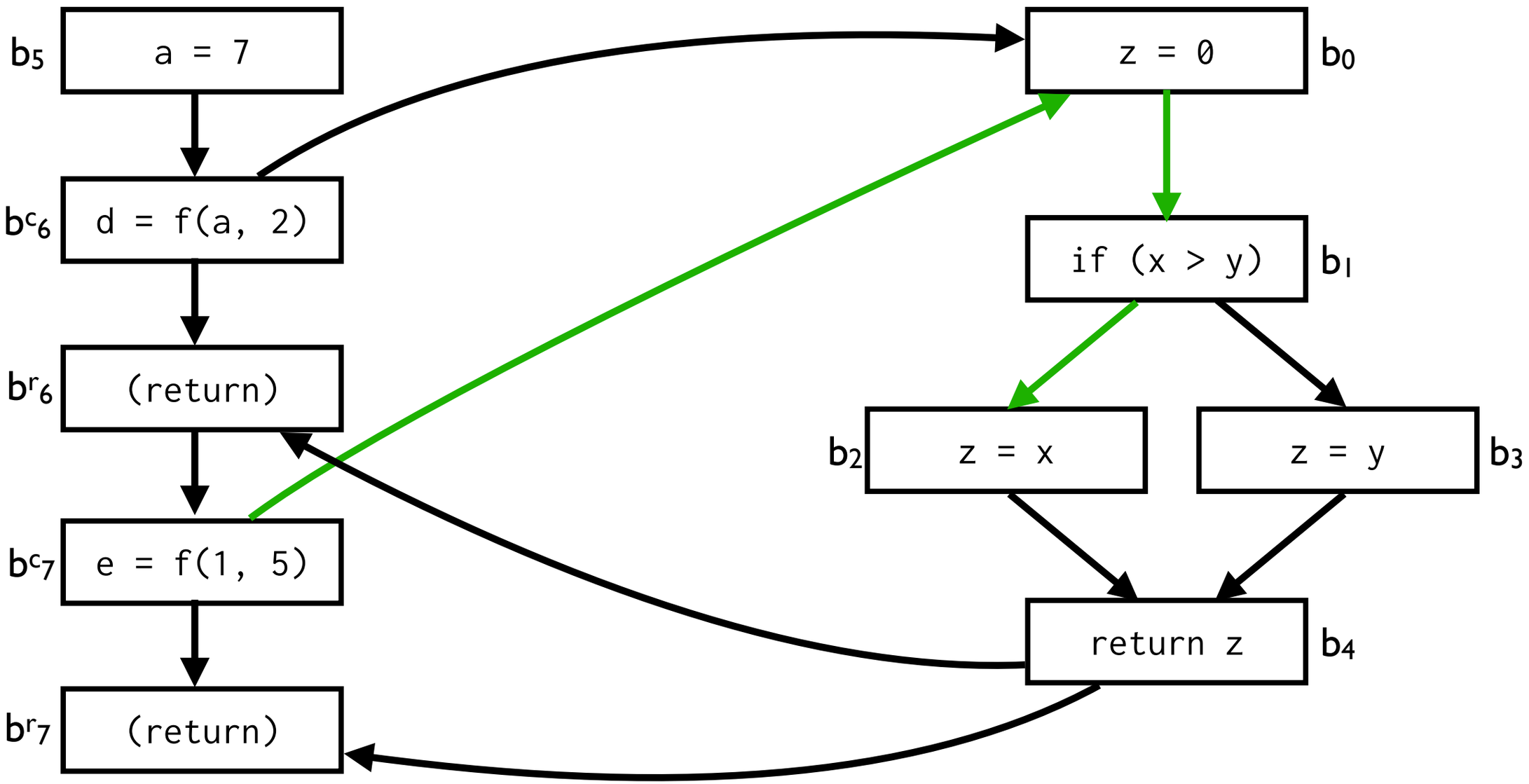
[b_5, b^c_6, b_0, b_1, b_3, b_4, b^r_6,]



Valid Paths

$[b_5, b^c_6, b_0, b_1, b_3, b_4, b^r_6]$

$[b^c_7, b_0, b_1, b_2]$



Valid Paths

- vpath_b: Set of all valid paths that reach b

$$\text{Path} ::= \langle P \rangle^* \langle M \rangle$$

$$P \quad ::= b_x \mid b^c_y$$

$$M \quad ::= b^c_x \langle M \rangle b^r_x \mid b_y \langle M \rangle \mid \langle M \rangle b_y \mid \varepsilon$$

- M productions: $b^c_x \langle M \rangle b^r_x$:

  call and return must **match up**

- P productions (prefix): b^c_y:

  can have **calls that we never return from**

Meet over Valid Paths (MVP)

- Intra-procedural analysis aims for MOP
  - Undecidable
- Now we aim for its **context-sensitive** version:

$$\mathbf{out}_{bi} = \bigsqcup_{[p0, \ldots pk] \in \mathbf{vpath}_{bi}} \mathbf{trans}_{p0} \circ \ldots \circ \mathbf{trans}_{pk} \circ \mathbf{trans}_{bi} (\bot)$$

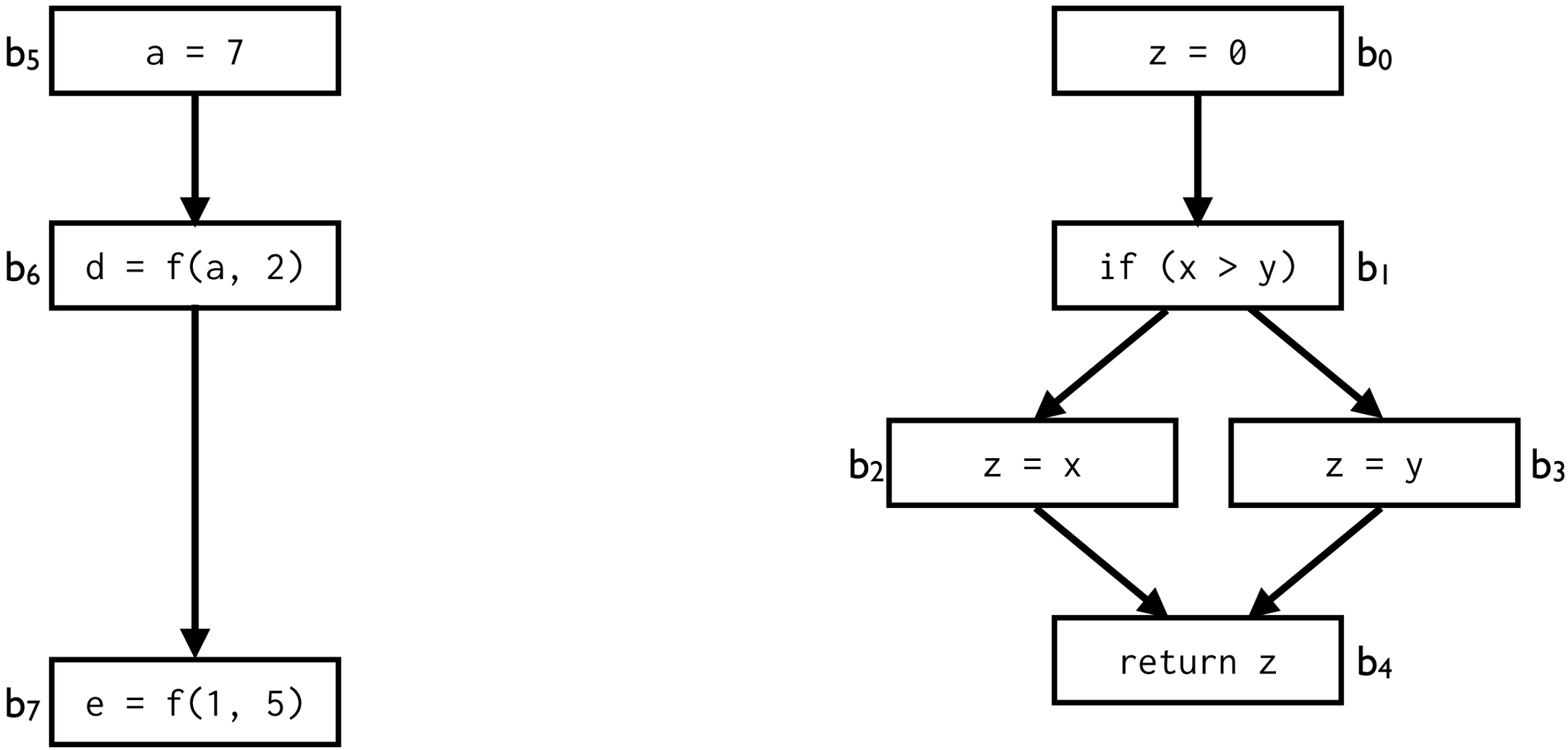
  - Undecidable, like MOP

Context Sensitivity

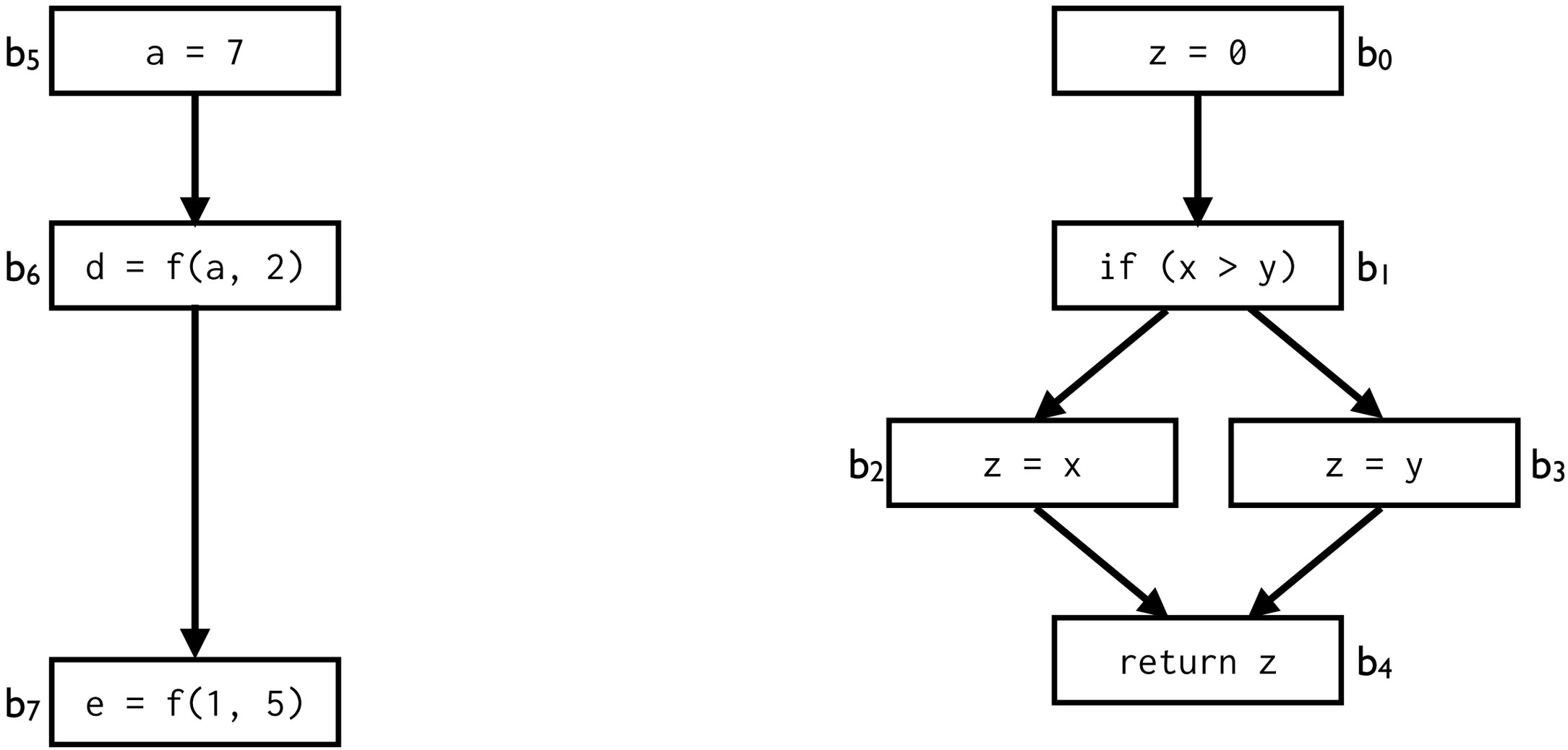
Context-Sensitivity

- Goal: distinguish among several context calls.
- Techniques:
  - Cloning
  - Inlining
  - Call Stacks/Call Strings
  - Procedure Summaries

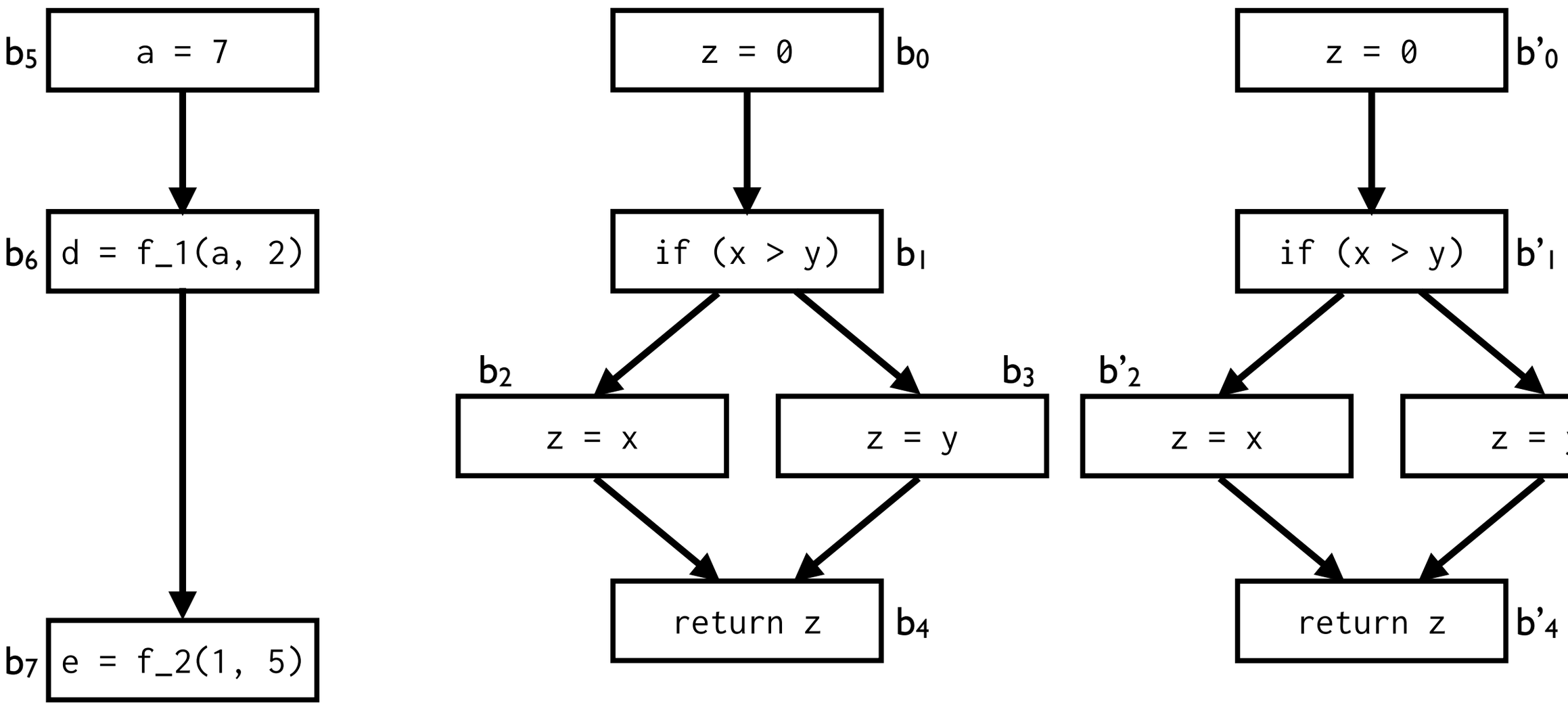
Context Sensitivity: Cloning



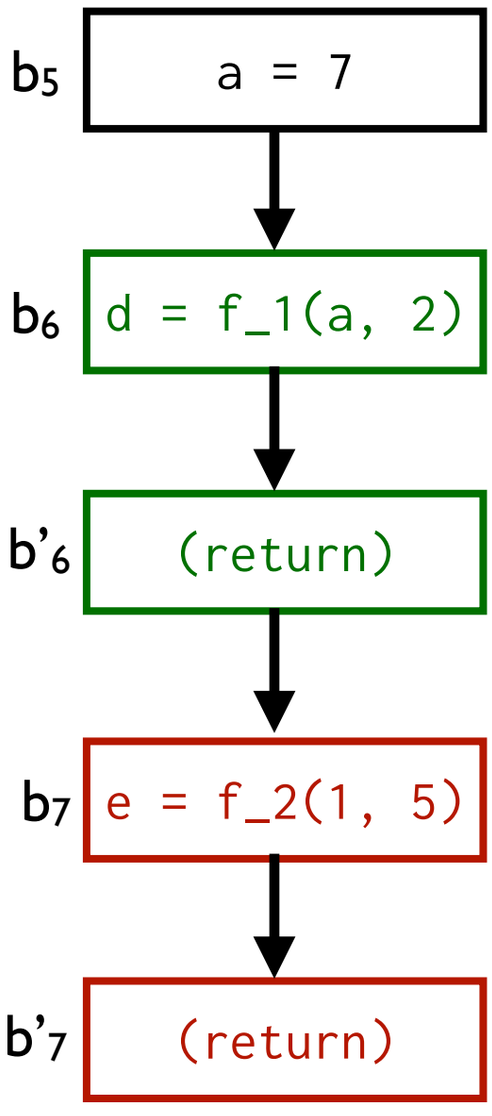
Context Sensitivity: Cloning

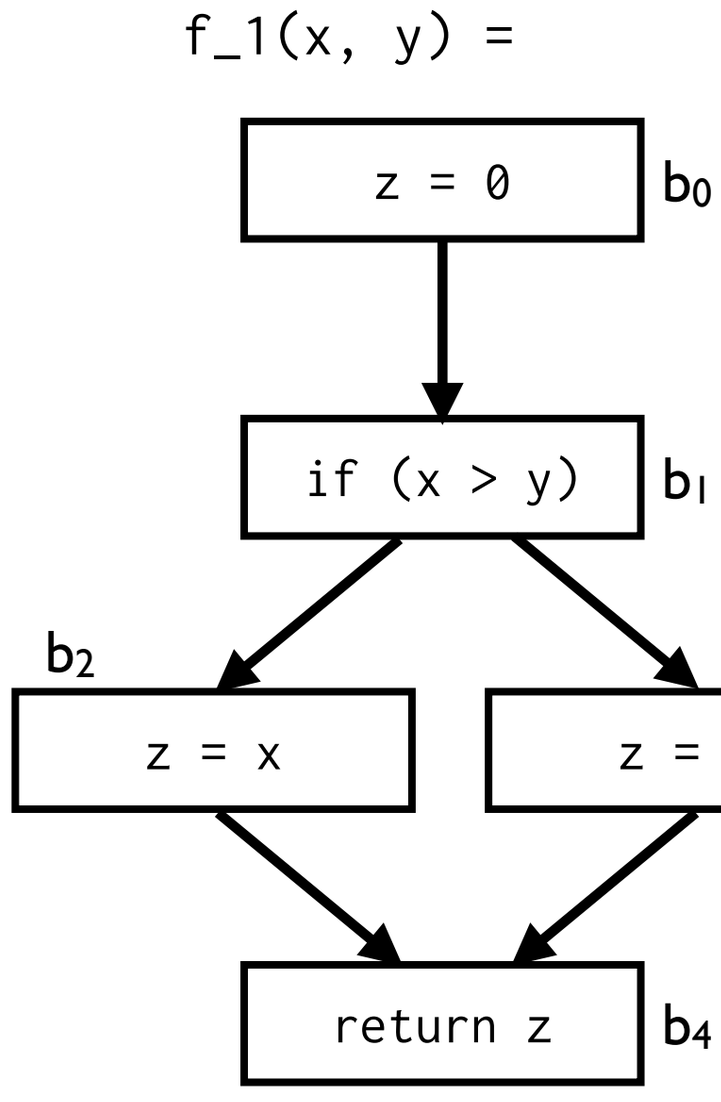


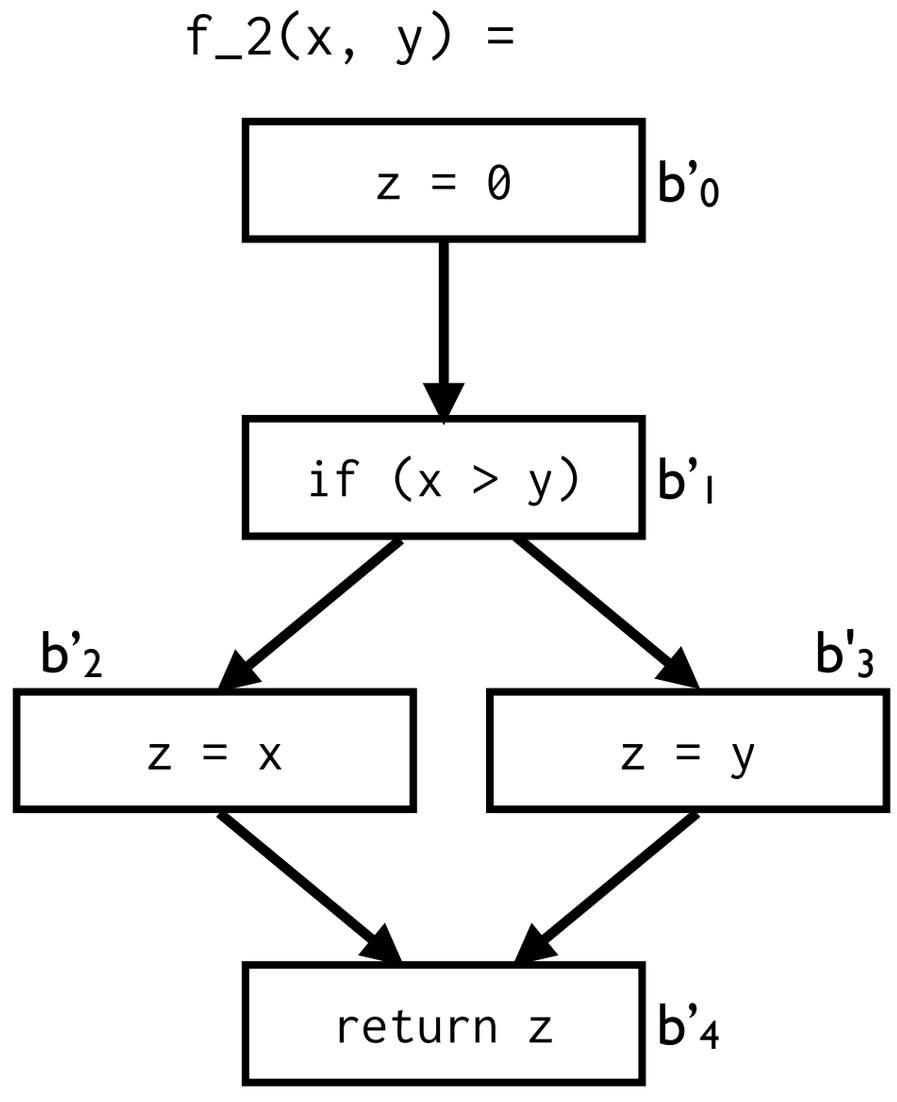
Context Sensitivity: Cloning



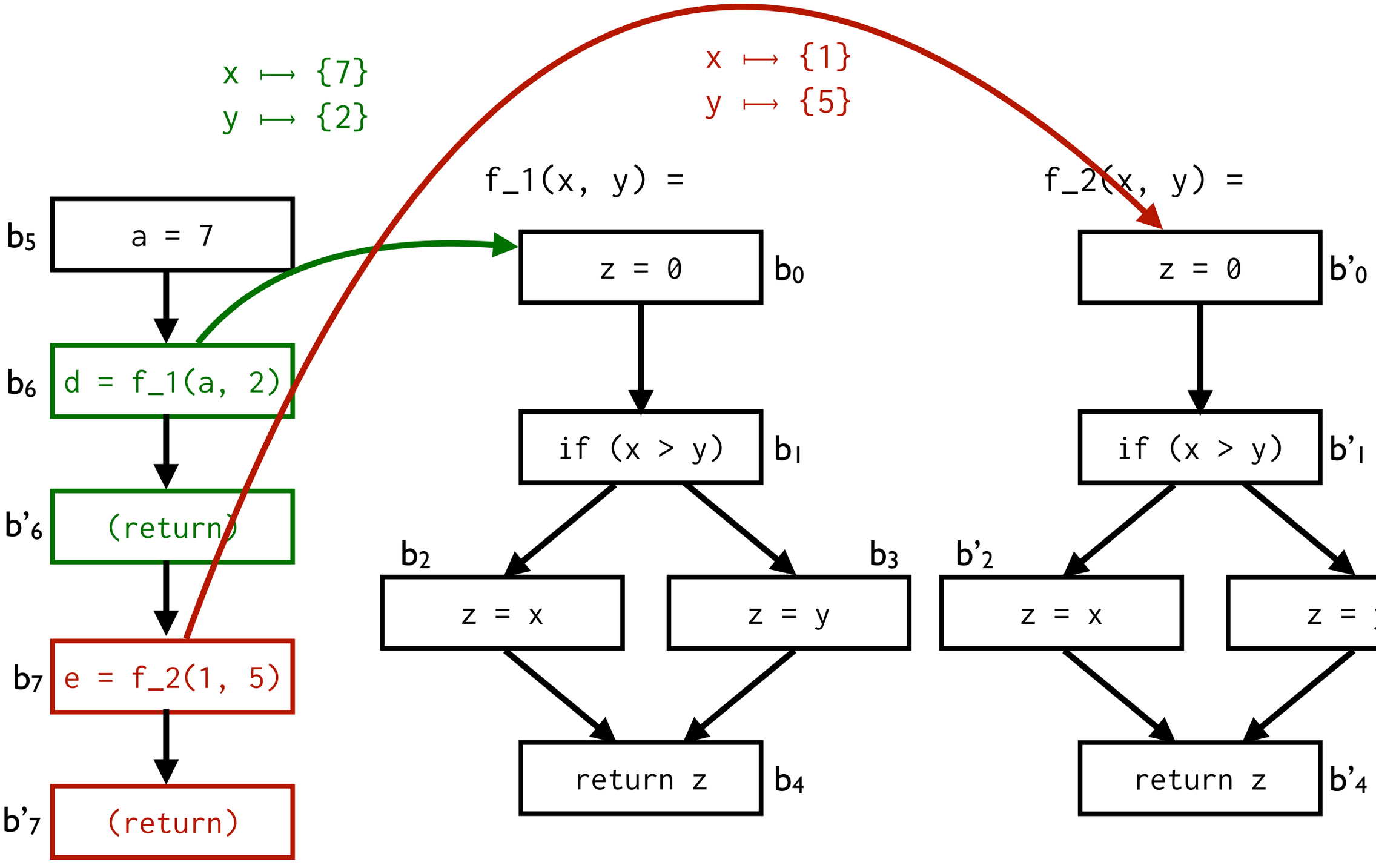
Context Sensitivity: Cloning



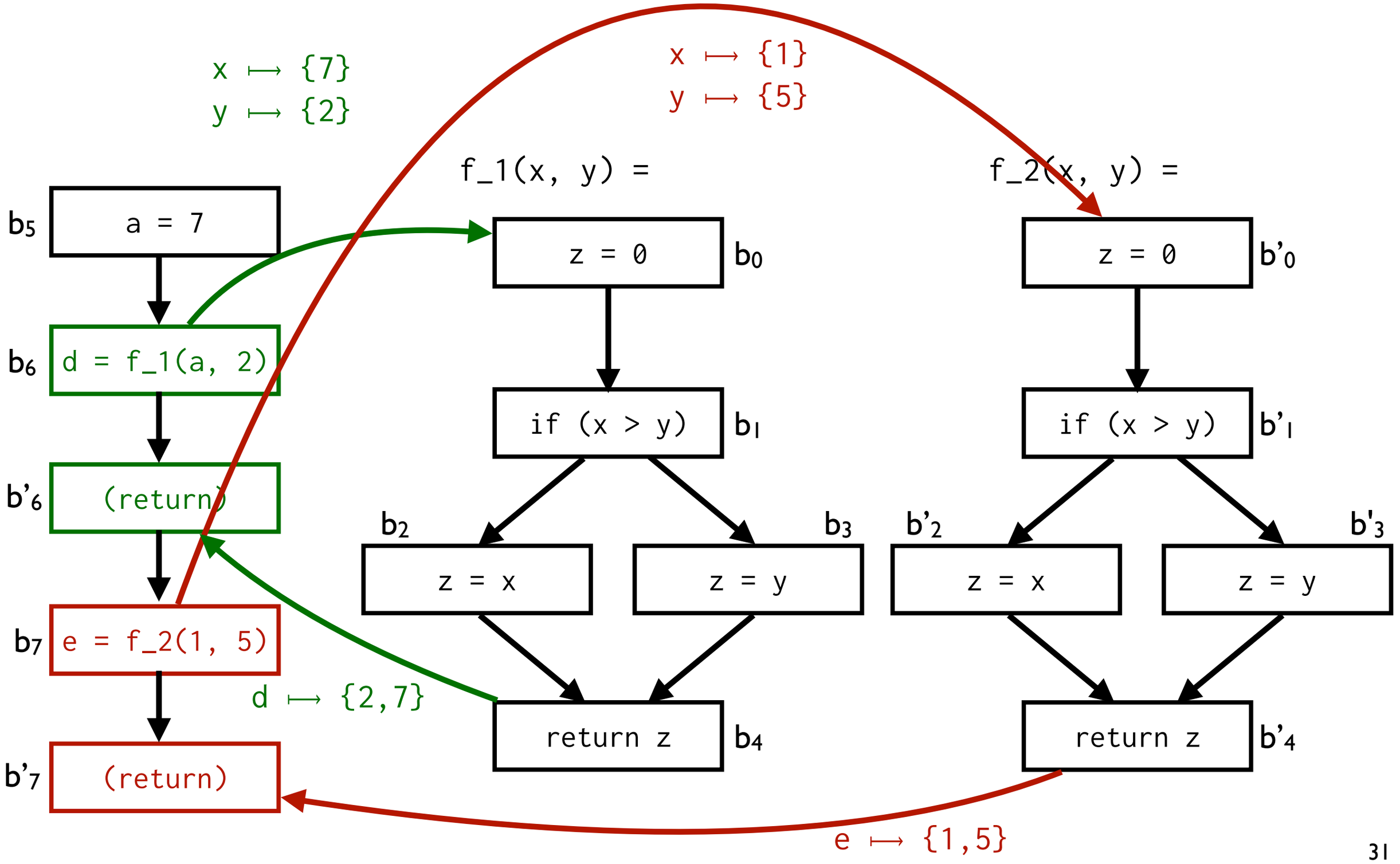




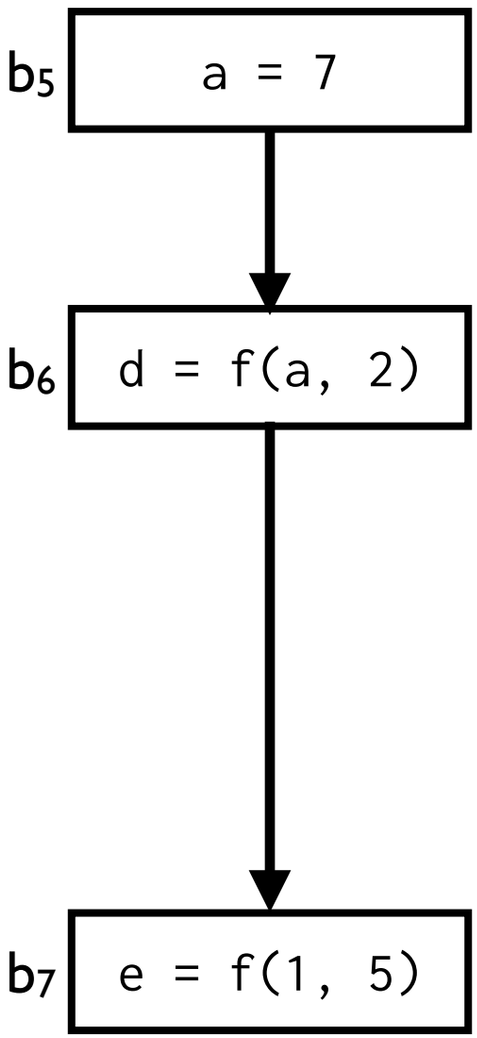
Context Sensitivity: Cloning

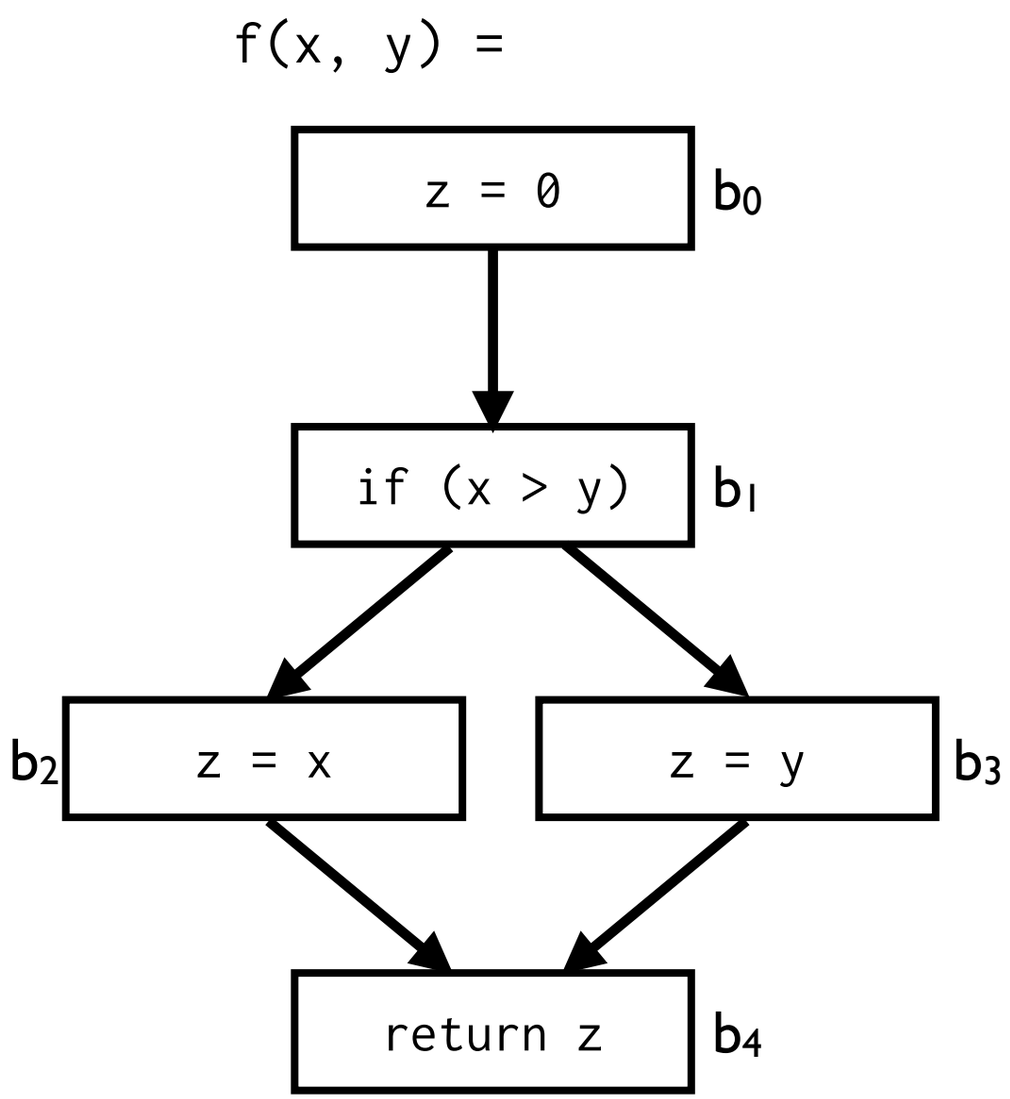


Context Sensitivity: Cloning

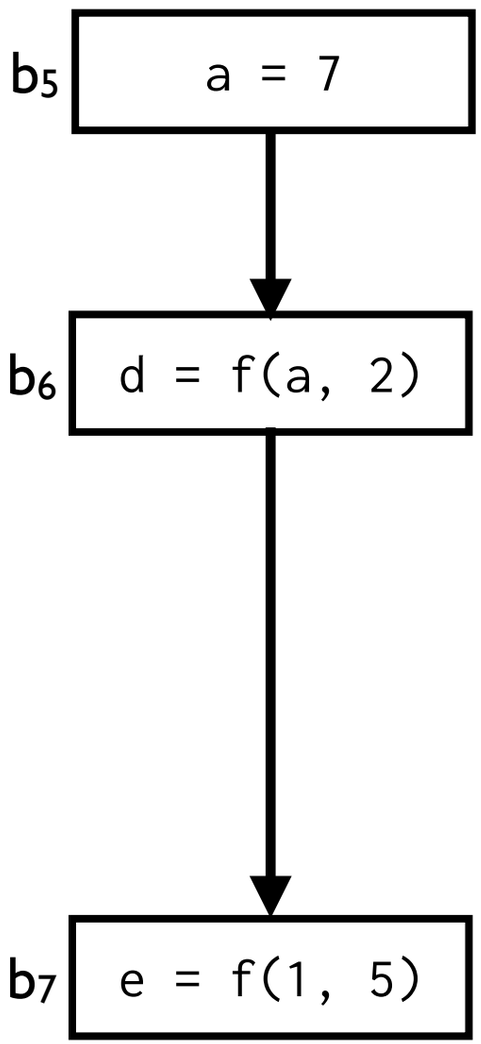


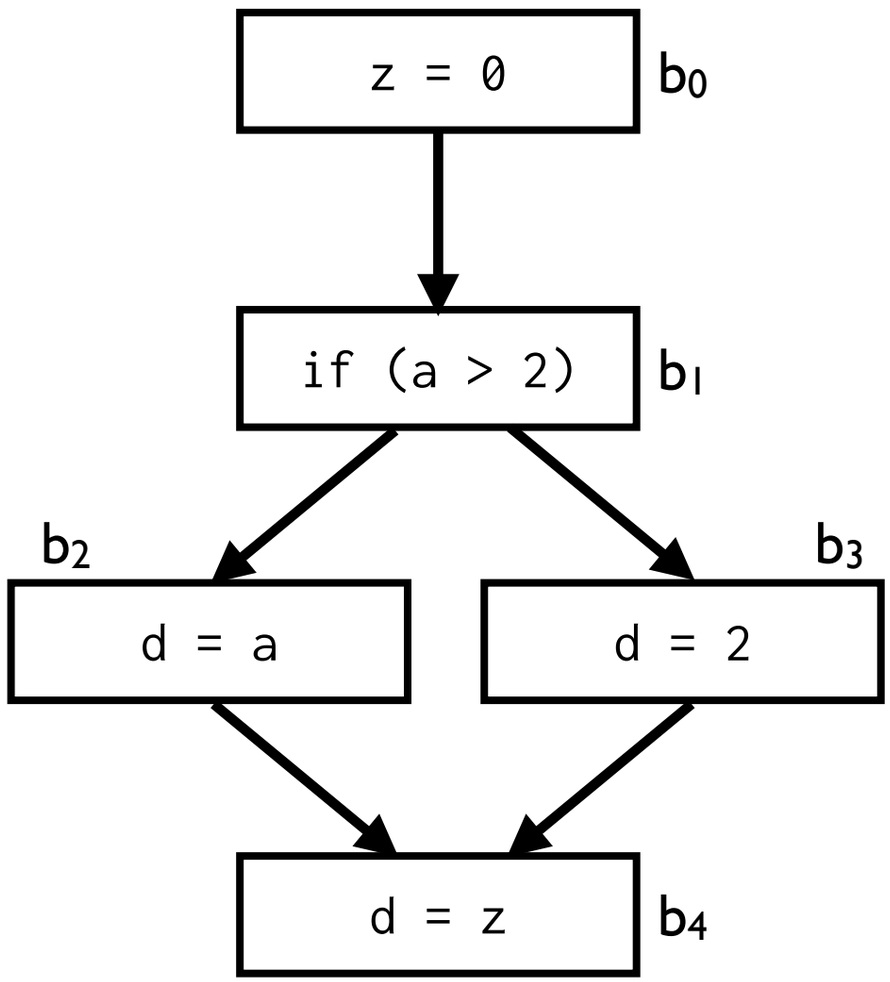
Context Sensitivity: Inlining

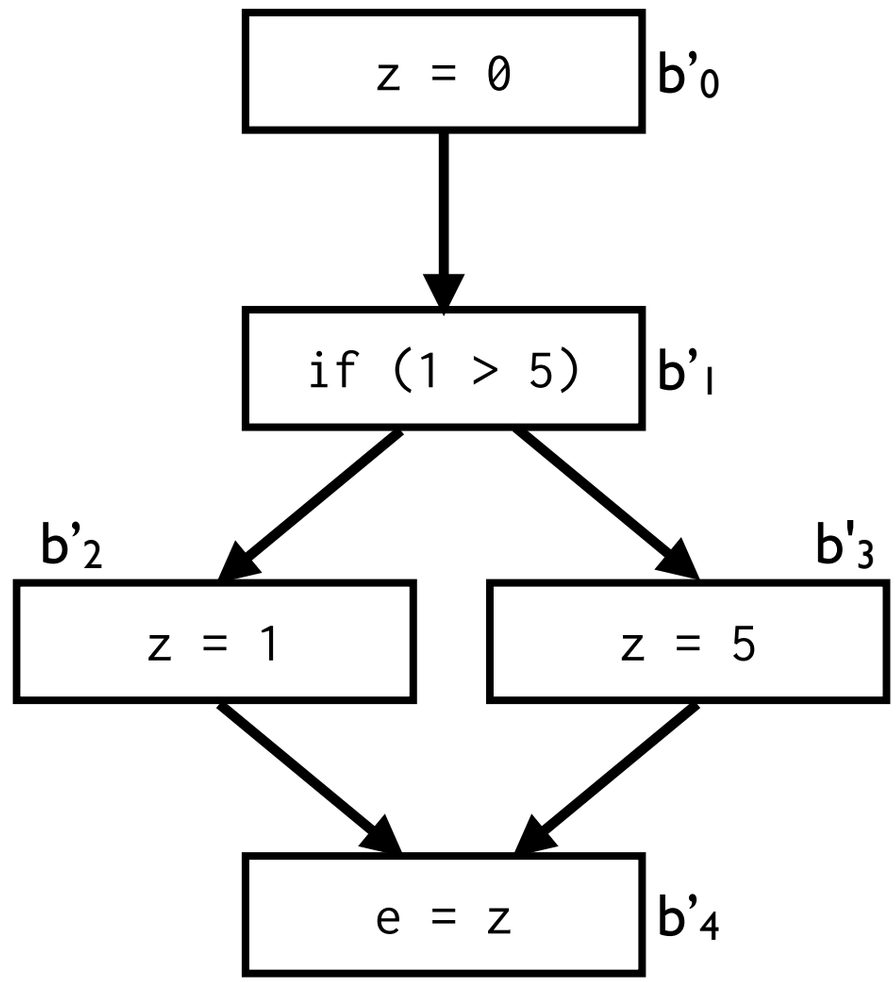




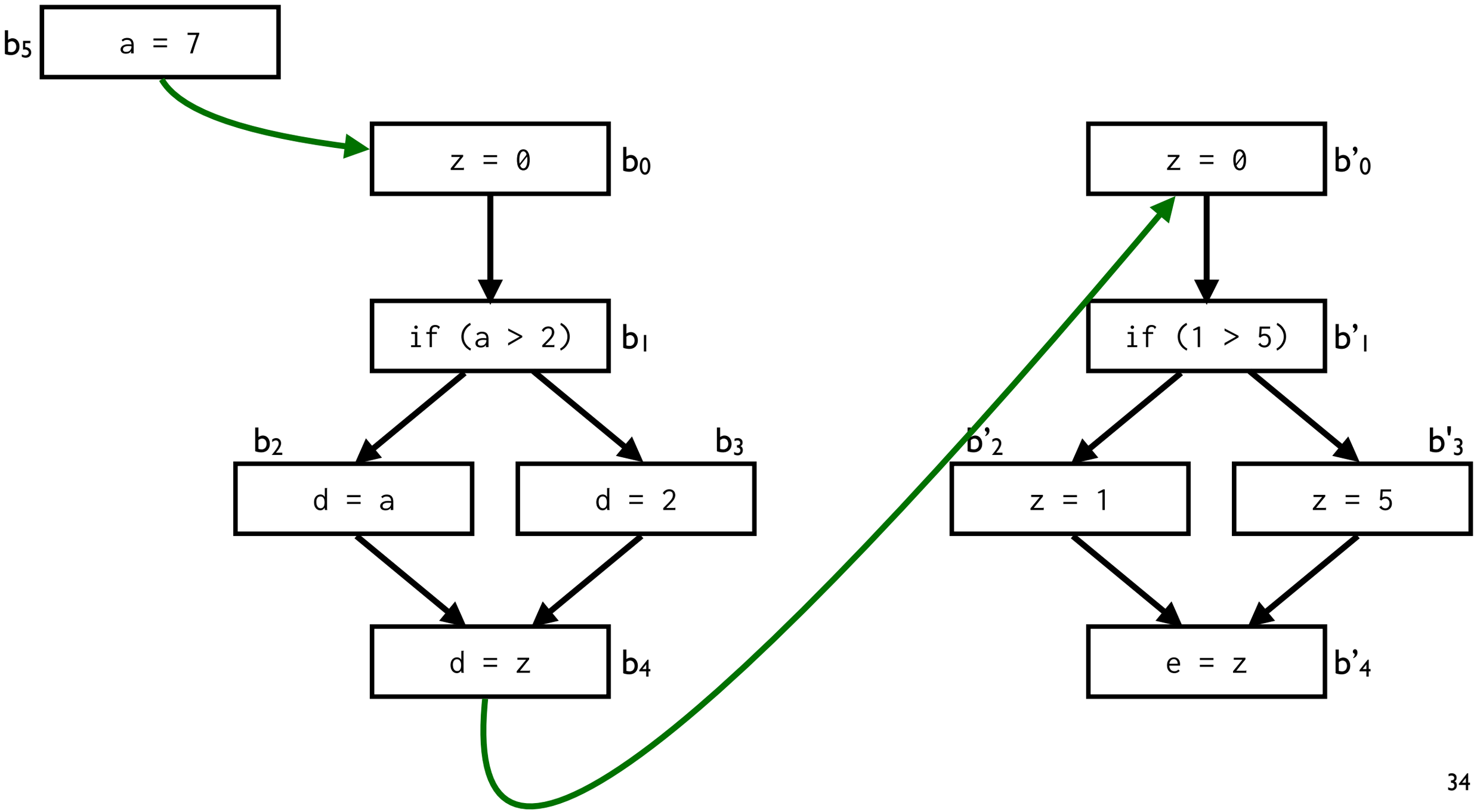
Context Sensitivity: Inlining



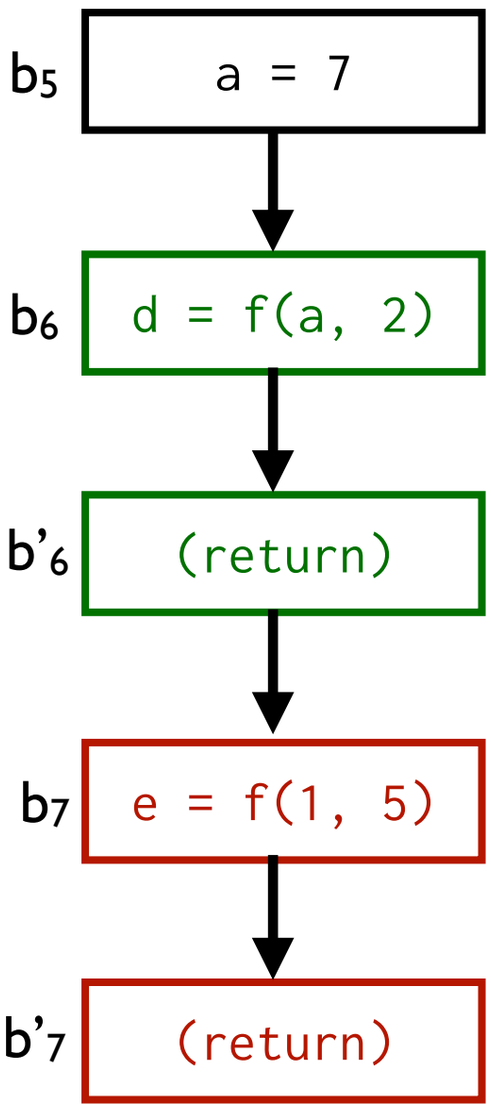


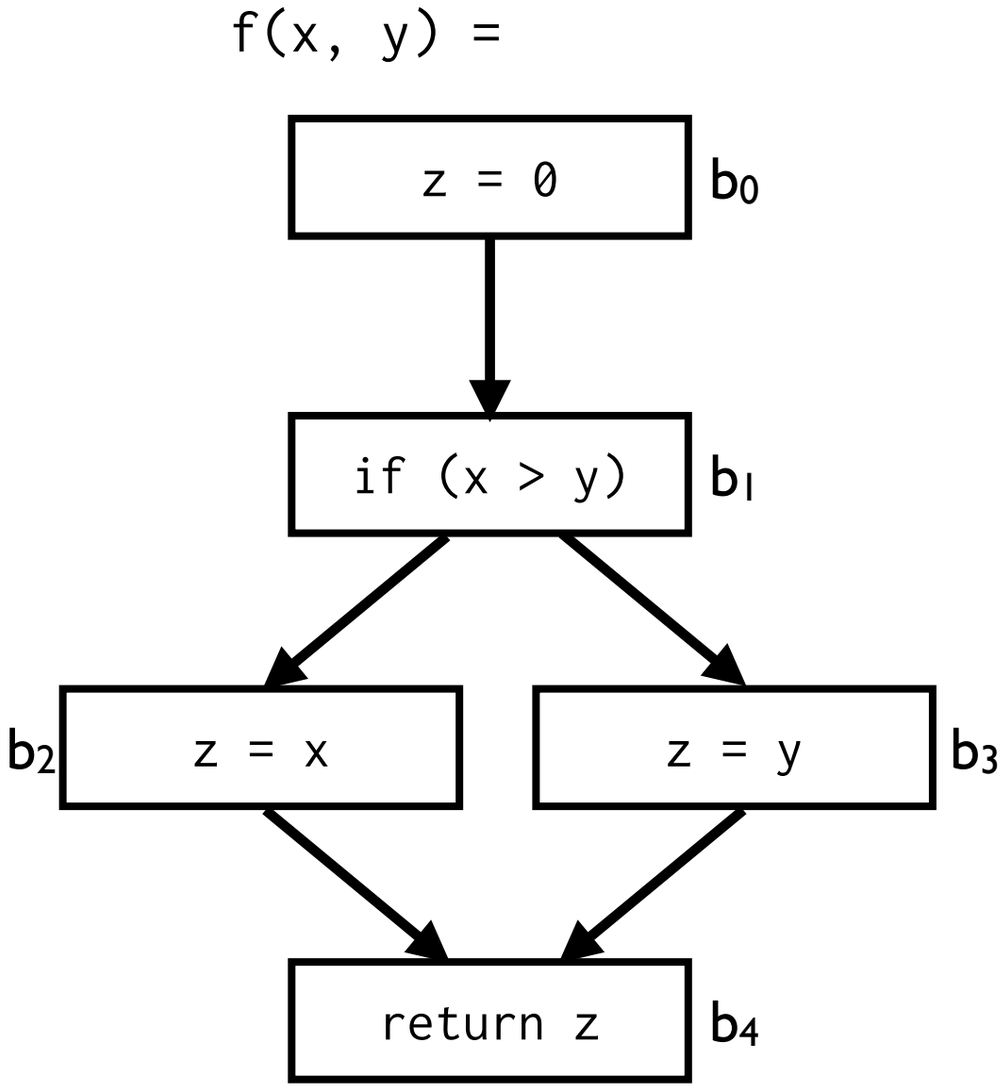


Context Sensitivity: Inlining

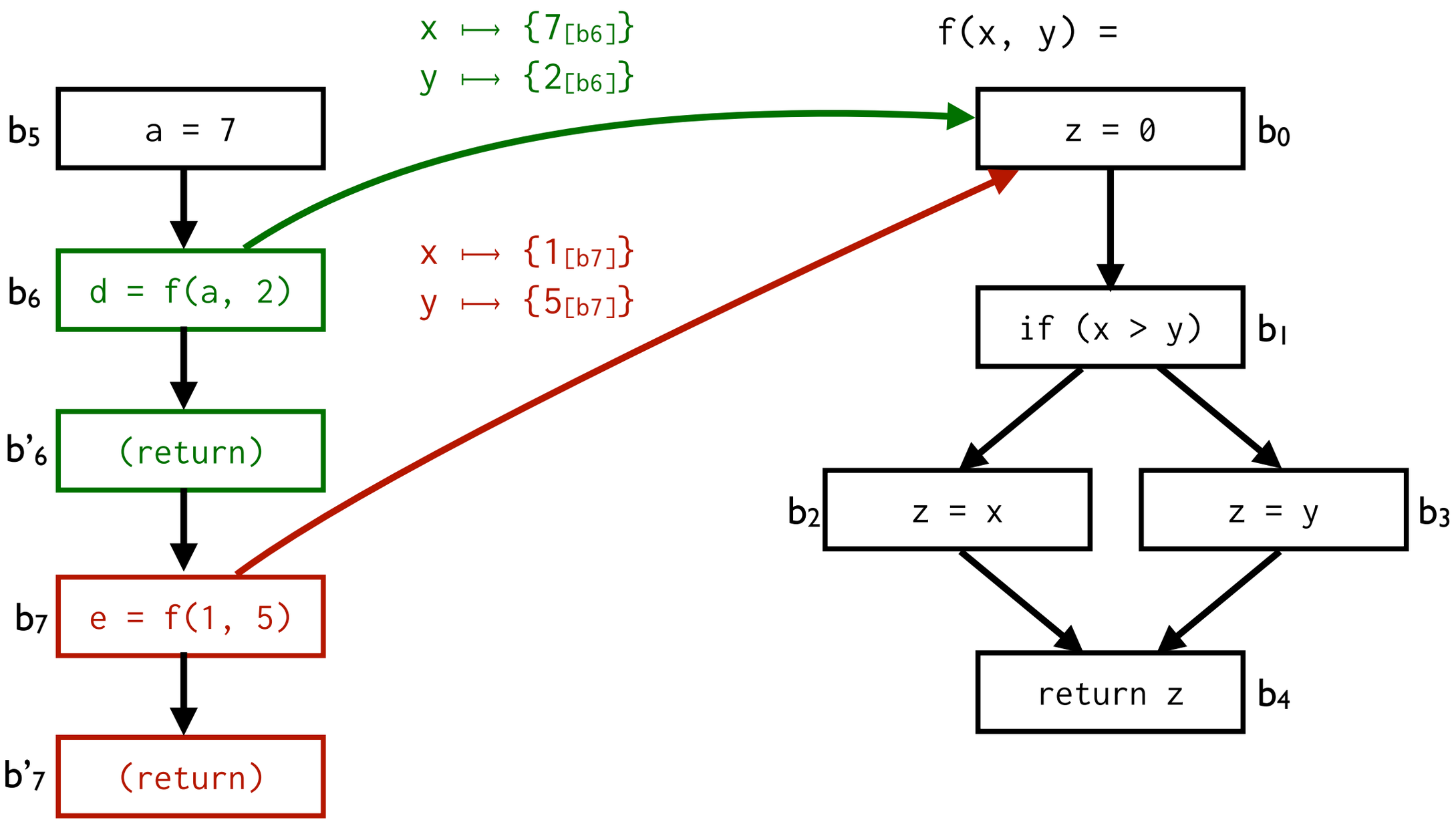


Context Sensitivity: Call Strings

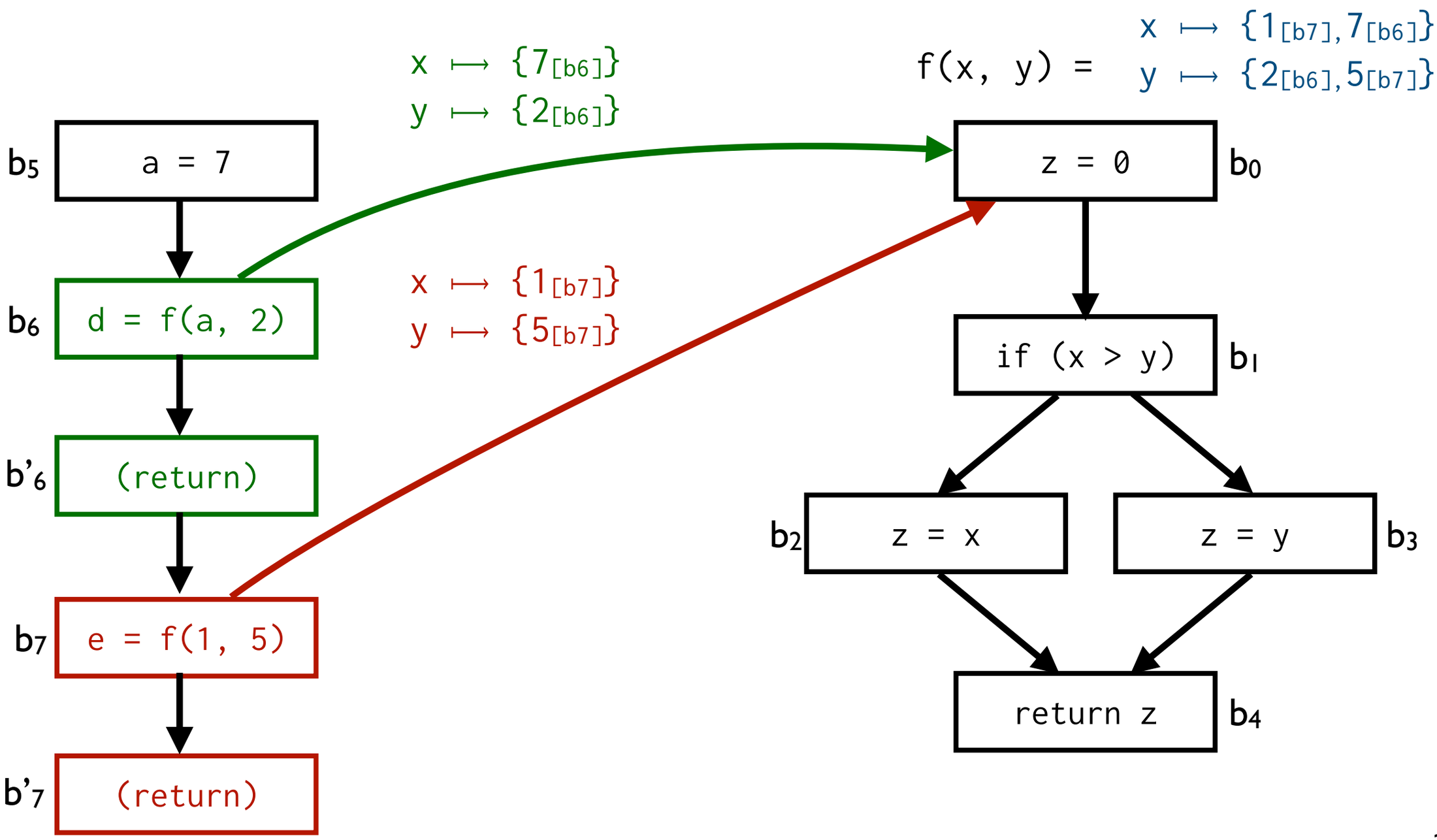




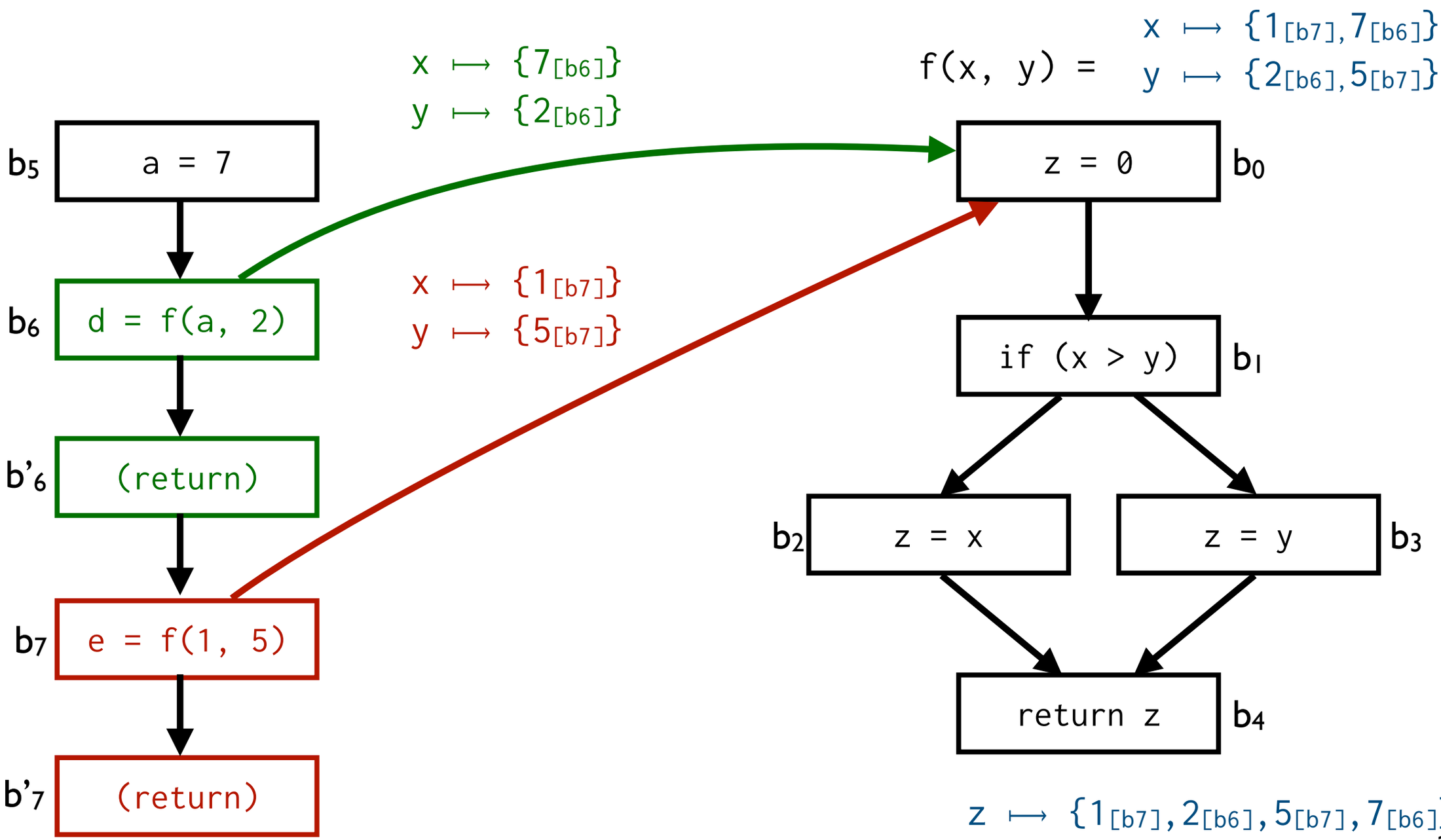
Context Sensitivity: Call Strings



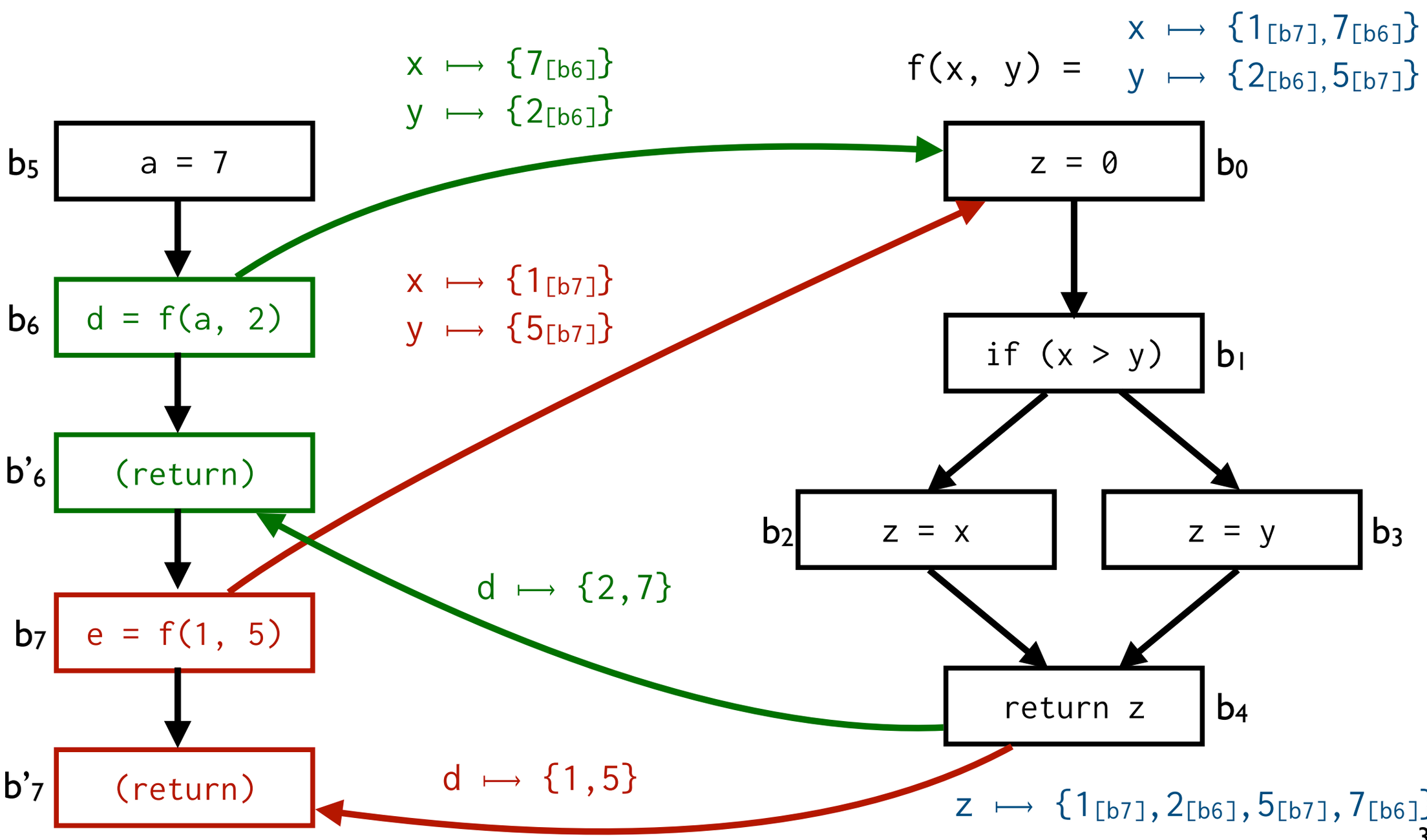
Context Sensitivity: Call Strings



Context Sensitivity: Call Strings



Context Sensitivity: Call Strings



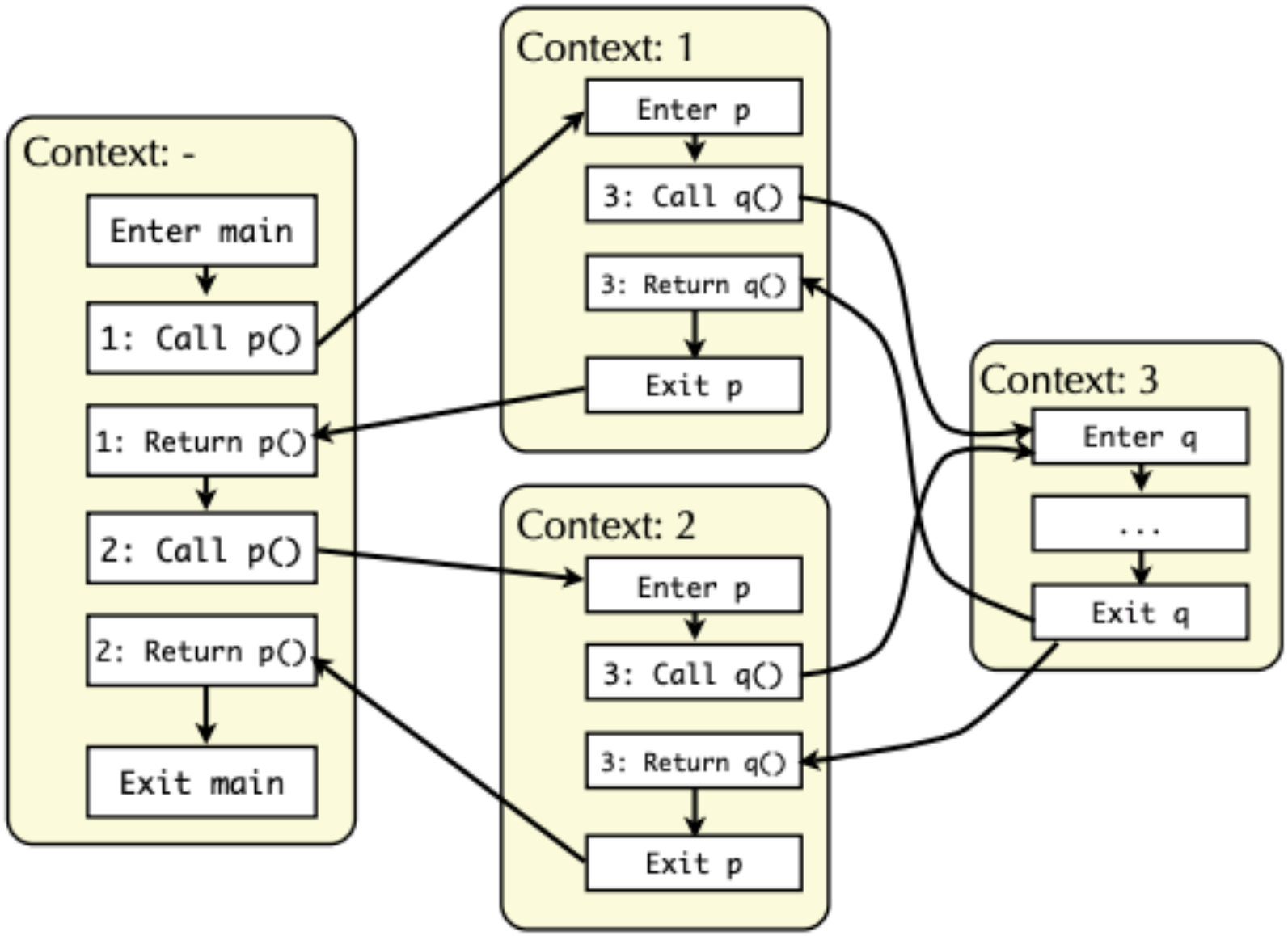
Context Sensitivity: Call Strings

- We used call sites to make context explicit
- Generalization:
- Call Strings support deeper nesting
- Examples: [b0, b6], [b1, b6]
- Must bound length of call strings to ensure termination

```
int g(y) {
  return y;
}

int f(x) {
  return g(x)
       + g(5);
}

…

f(1);
f(2);
```

Context Sensitivity: Call Strings

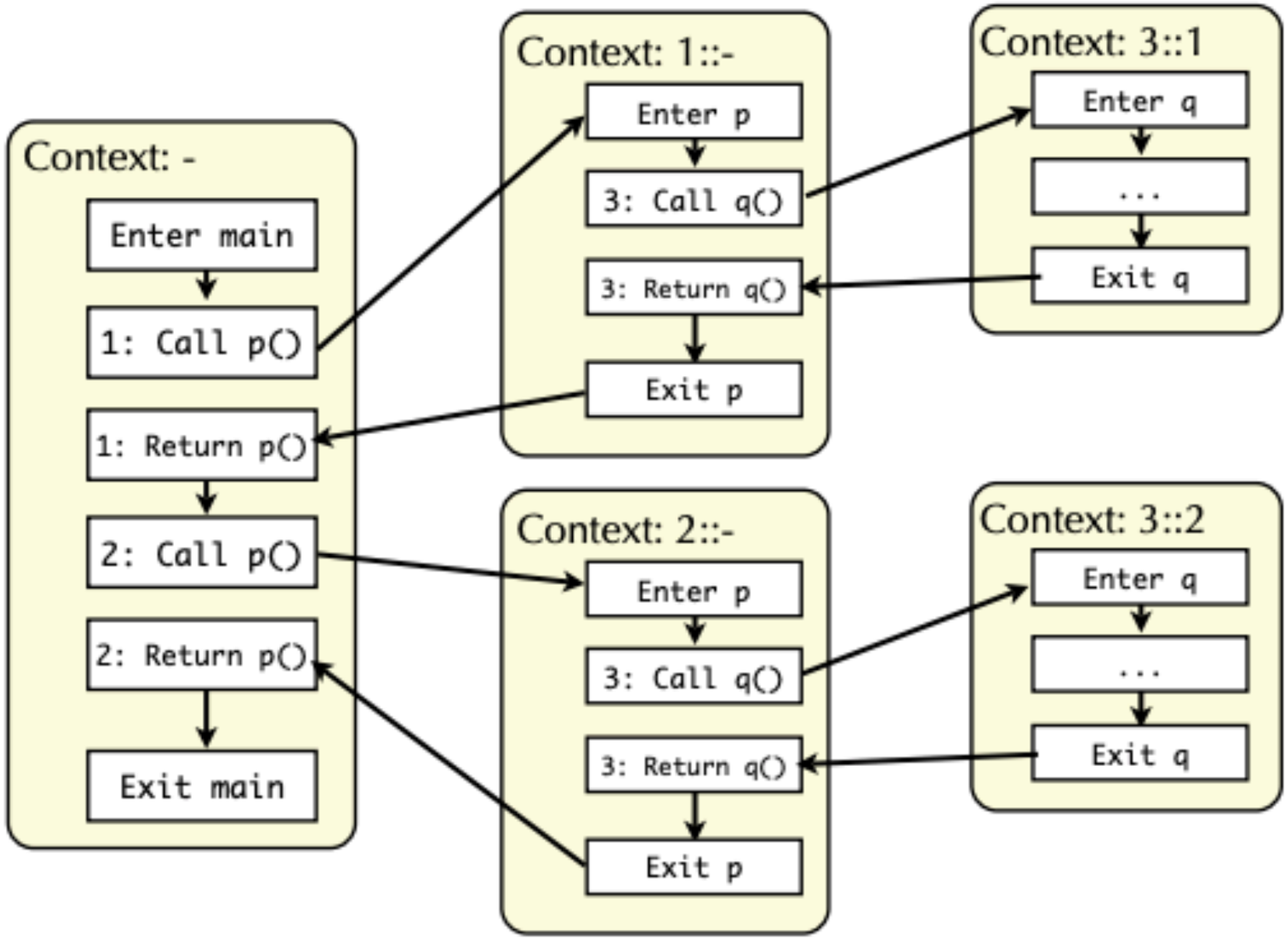
```
main() {
  1: p();
  2: p();
}

p() {
  3: q();
}
q() {
  …
}
```



Call strings of size 1

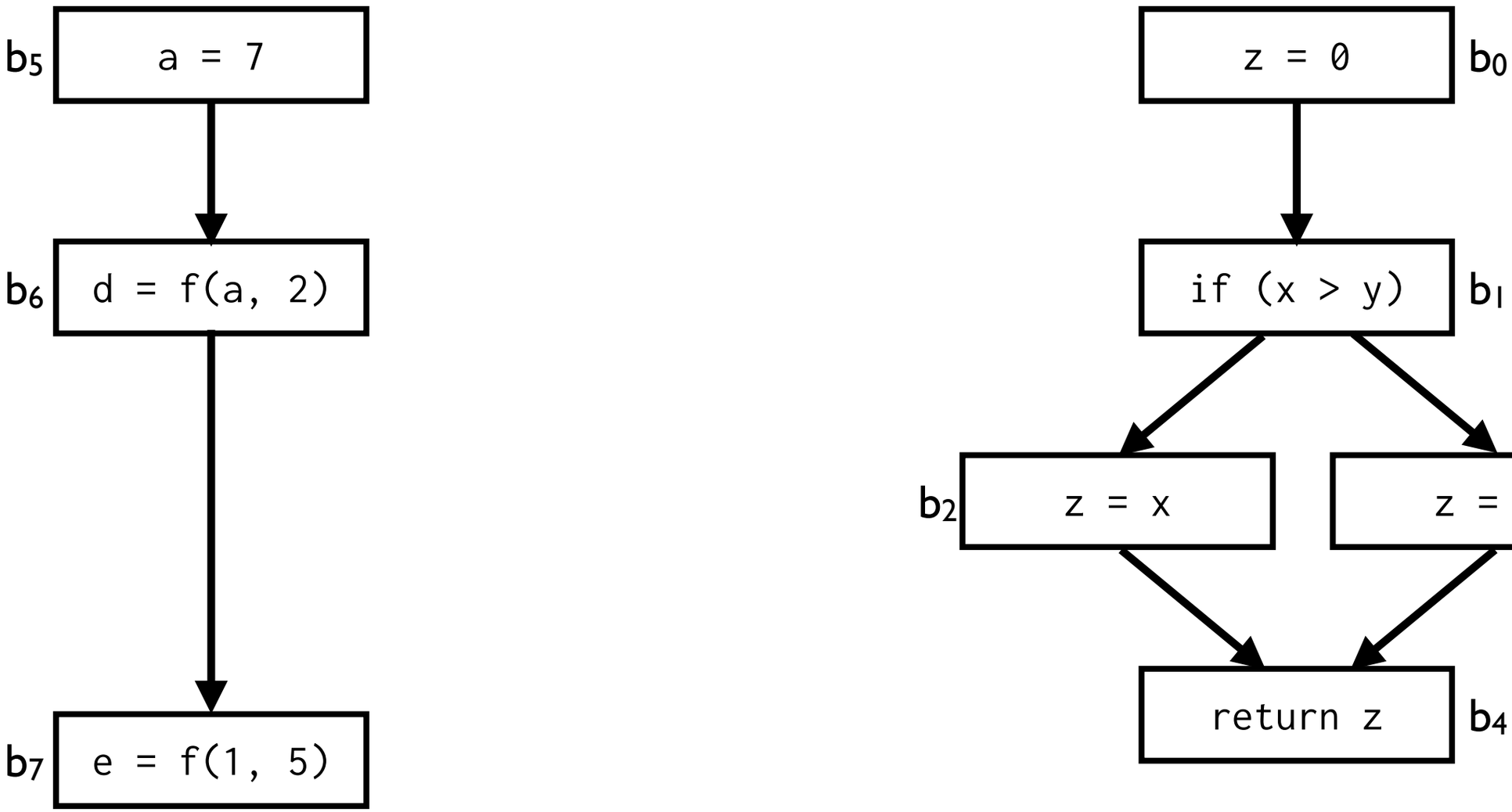
Context Sensitivity: Call Strings

```
main() {
  1: p();
  2: p();
}

p() {
  3: q();
}
q() {
  …
}
```

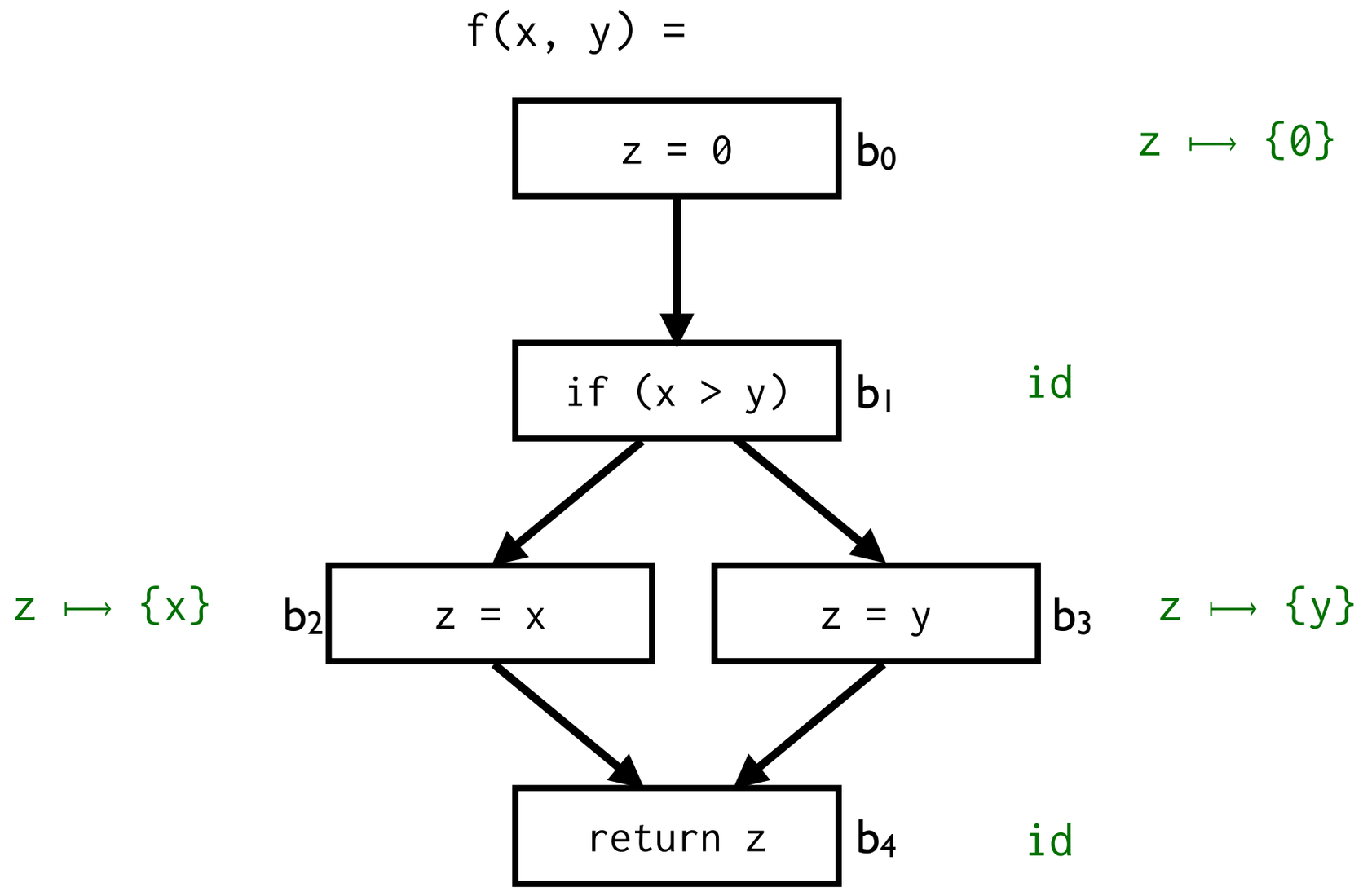


Call strings of size 2

Context Sensitivity: Procedure Summaries

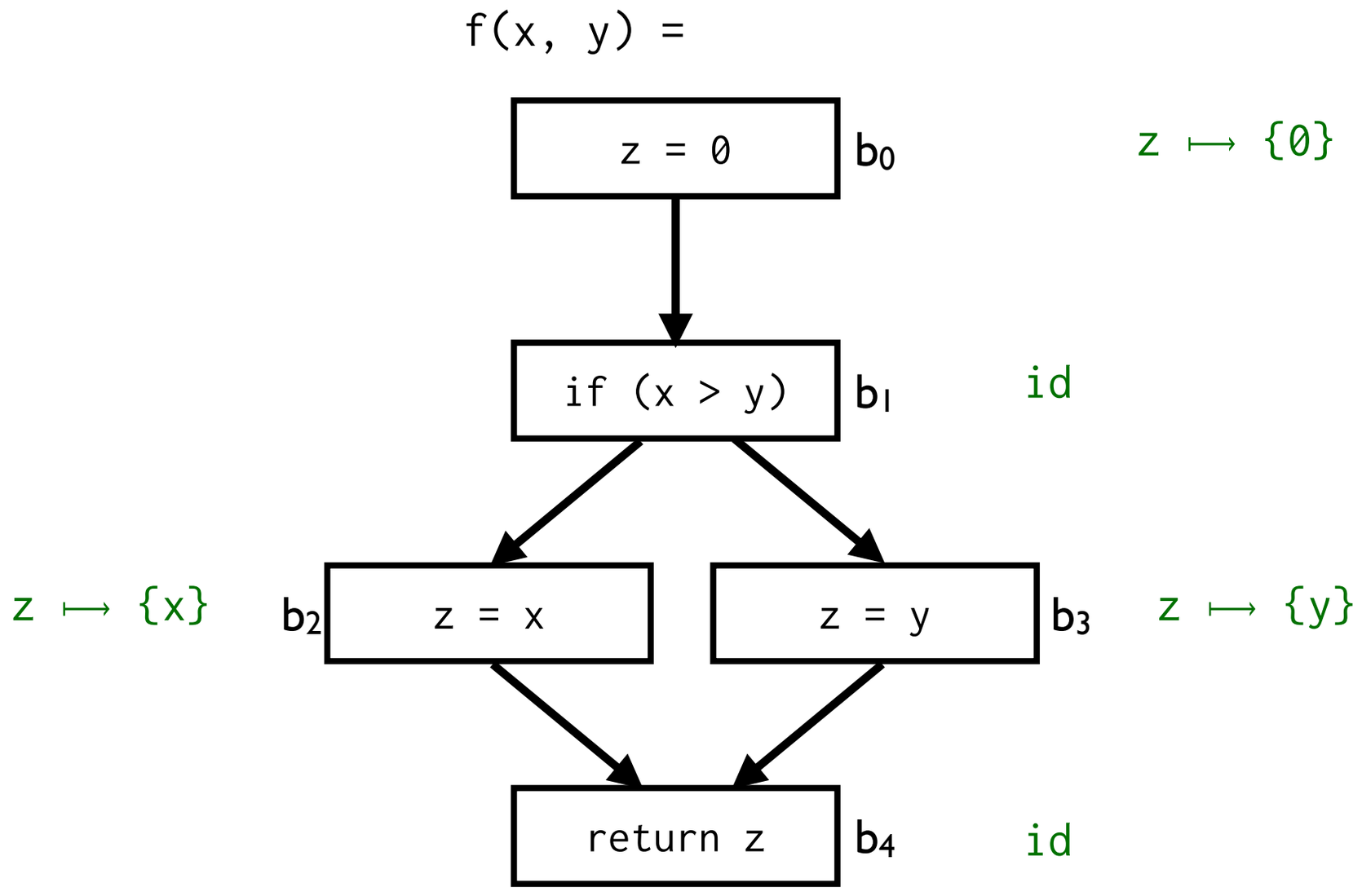


Composed Transfer Function



$trans_{b0} \circ trans_{b1} = [z \longmapsto 0]$

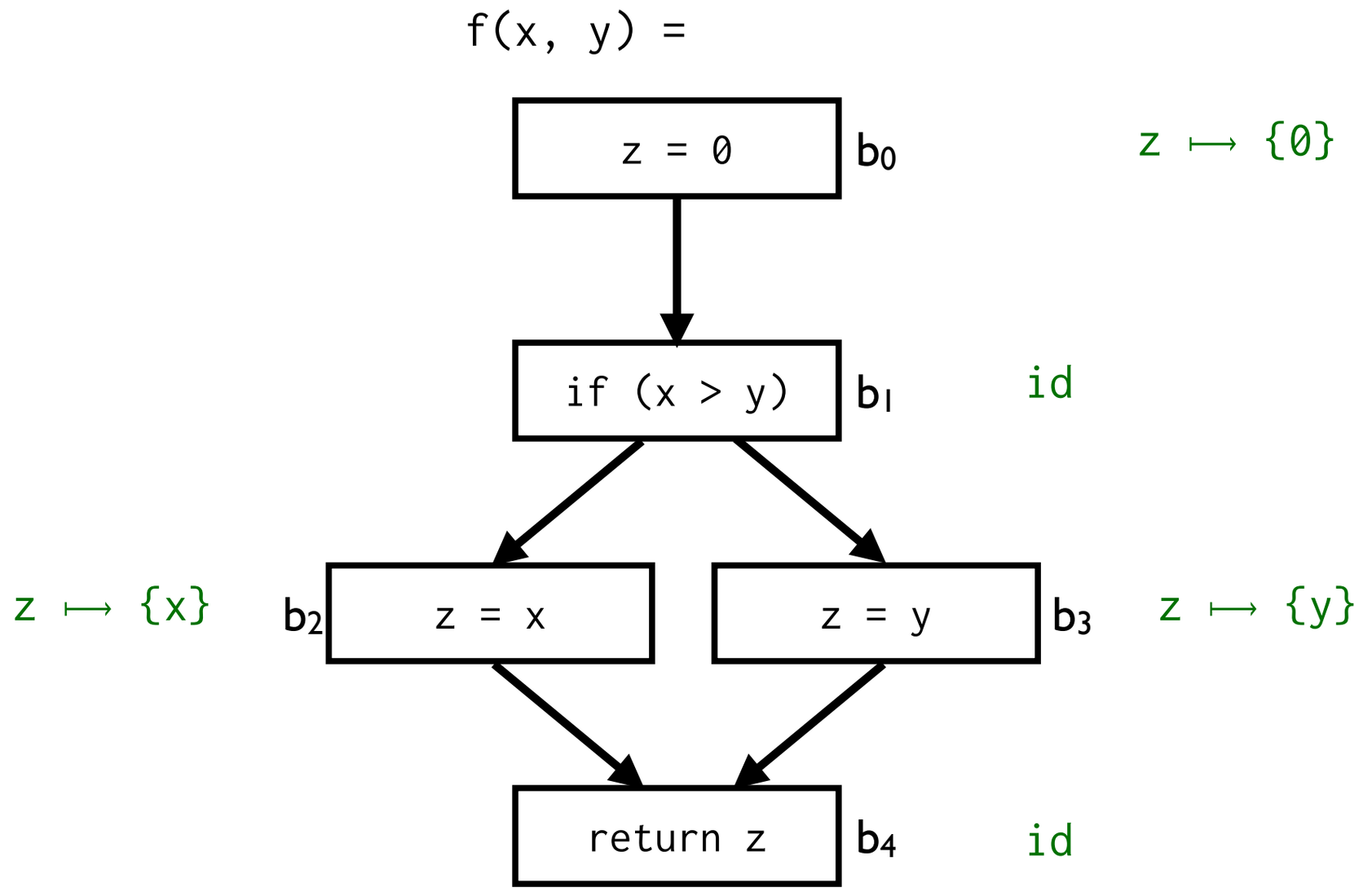
Composed Transfer Function



$trans_{b0} \circ trans_{b1} = [z \longmapsto 0]$

$trans_{b0} \circ trans_{b1} \circ trans_{b2} = [z \longmapsto \{x\}]$

Composed Transfer Function

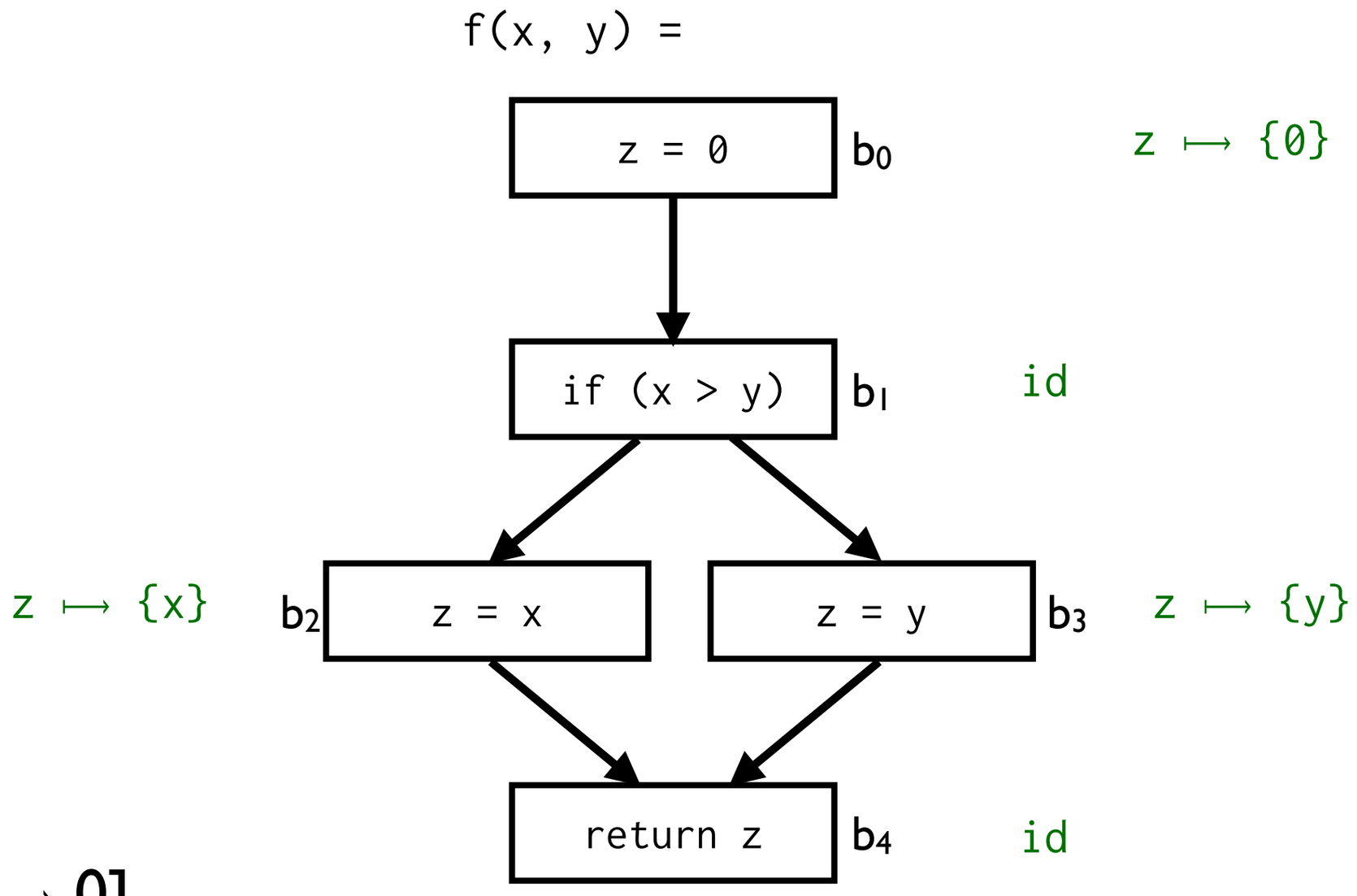


$trans_{b0} \circ trans_{b1} = [z \longmapsto 0]$

$trans_{b0} \circ trans_{b1} \circ trans_{b2} = [z \longmapsto \{x\}]$

$trans_{b0} \circ trans_{b1} \circ trans_{b3} = [z \longmapsto \{y\}]$

Composed Transfer Function



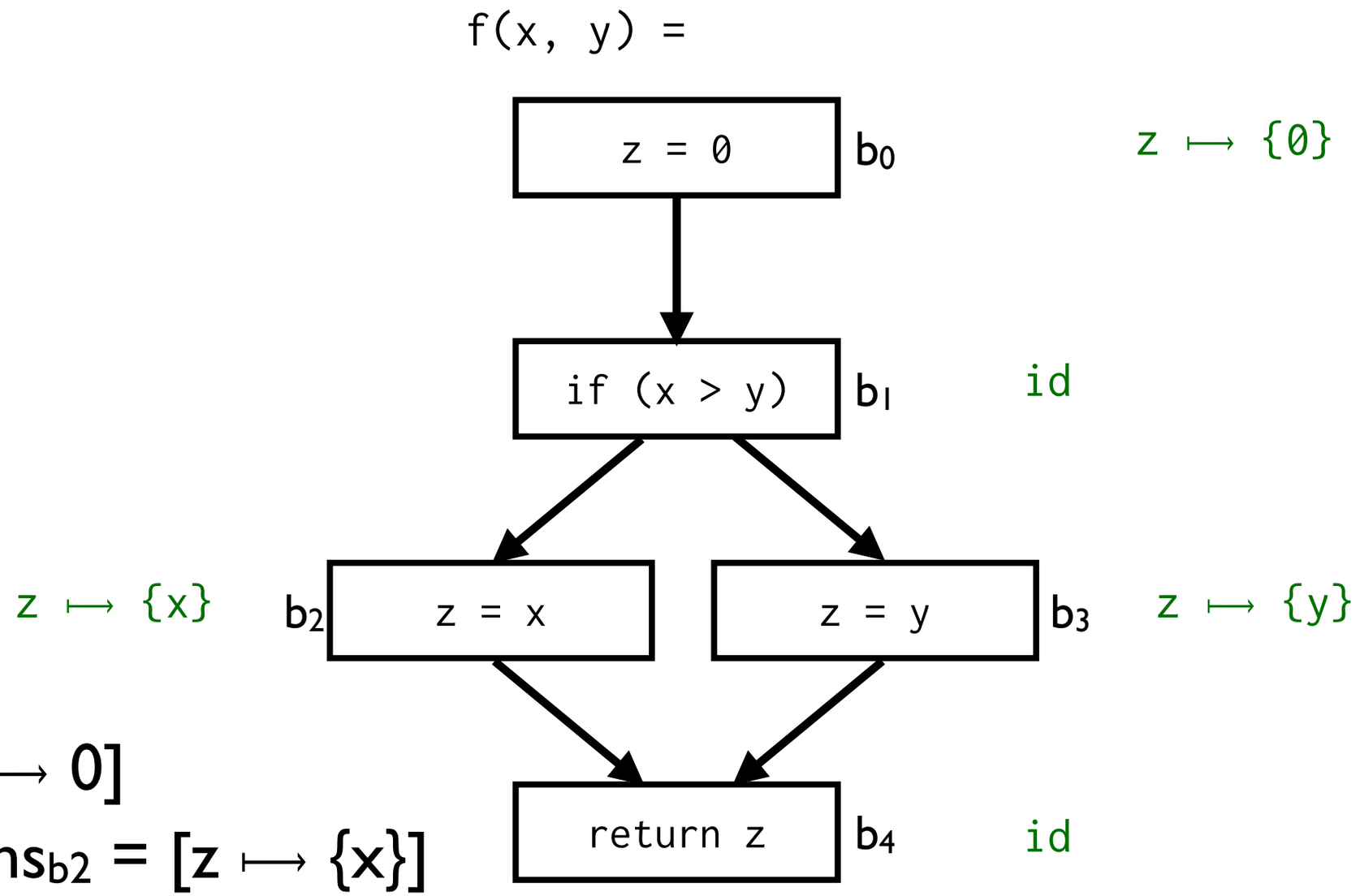
$trans_{b0} \circ trans_{b1} = [z \longmapsto 0]$

$trans_{b0} \circ trans_{b1} \circ trans_{b2} = [z \longmapsto \{x\}]$

$trans_{b0} \circ trans_{b1} \circ trans_{b3} = [z \longmapsto \{y\}]$

$trans_{b0} \circ trans_{b1} \circ (trans_{b2} \sqcup trans_{b3}) = [z \longmapsto \{x,y\}]$

Composed Transfer Function



$trans_{b0} \circ trans_{b1} = [z \longmapsto 0]$

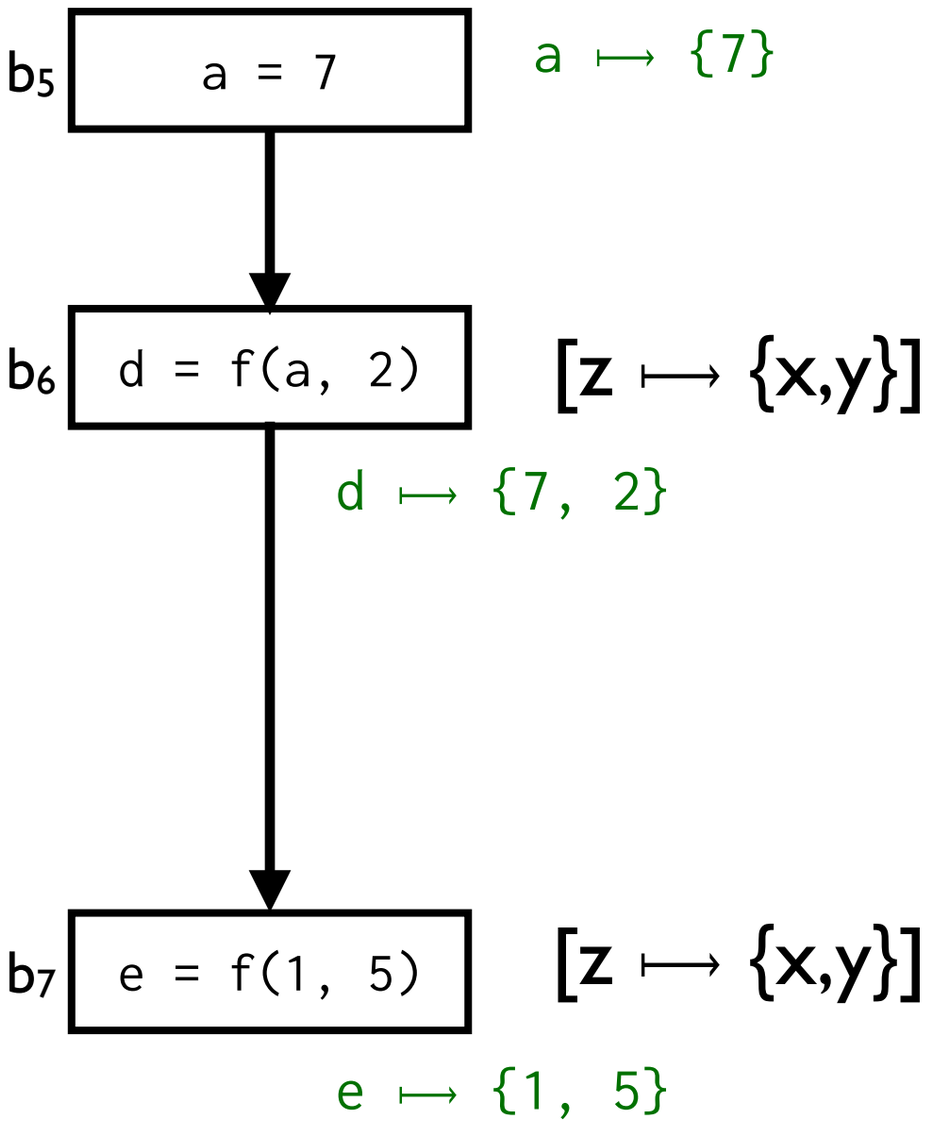
$trans_{b0} \circ trans_{b1} \circ trans_{b2} = [z \longmapsto \{x\}]$

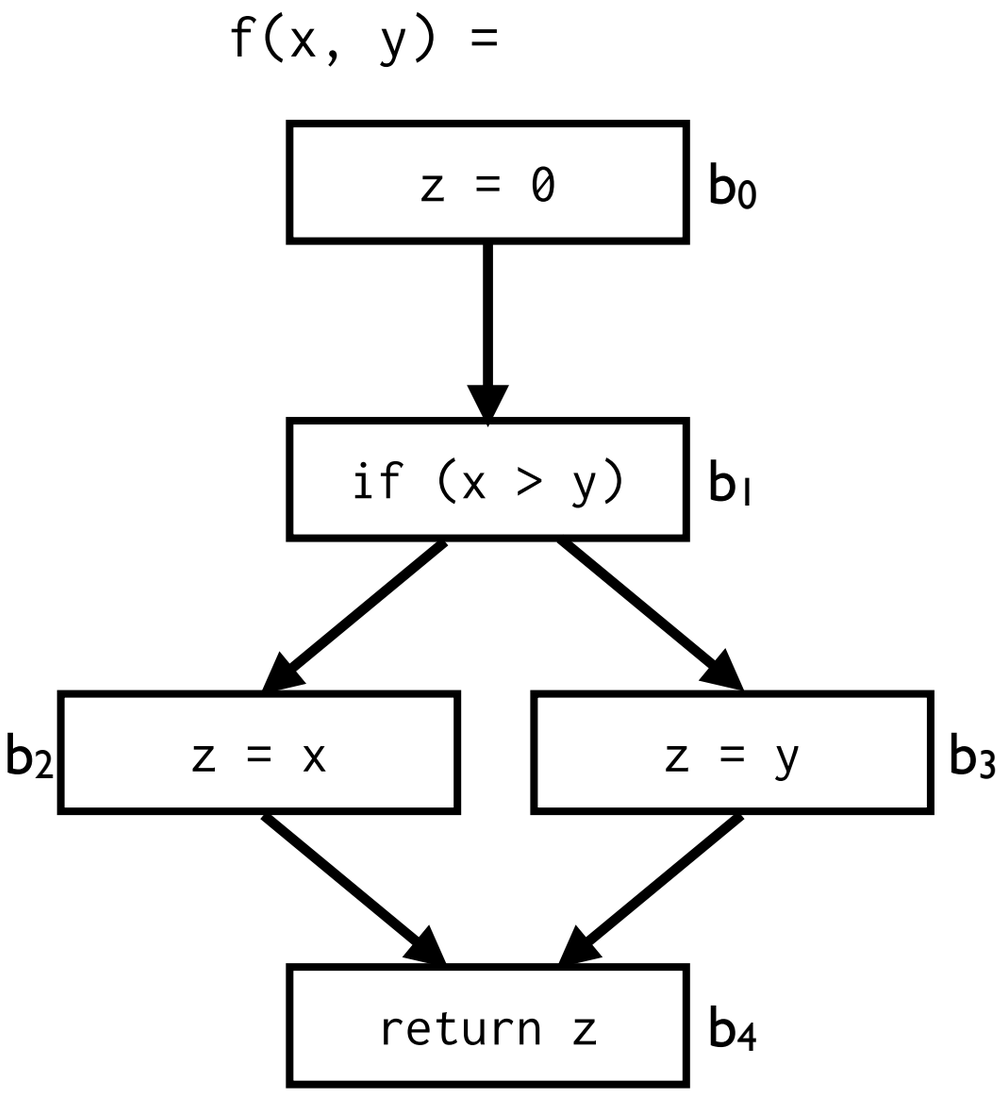
$trans_{b0} \circ trans_{b1} \circ trans_{b3} = [z \longmapsto \{y\}]$

$trans_{b0} \circ trans_{b1} \circ (trans_{b2} \sqcup trans_{b3}) = [z \longmapsto \{x,y\}]$

$trans_{b0} \circ trans_{b1} \circ (trans_{b2} \sqcup trans_{b3}) \circ trans_{b4} = [z \longmapsto \{x,y\}]$

Context Sensitivity: Procedure Summaries





Composed Transfer Function

- Symbolically composing transfer functions yields a combined transfer function for f():
  - $trans_f$ = [`return` ⟼ {x,y}]
- Can use trans at each call site of f() without knowing the subroutine's body

Composed Transfer Function

- Limitations:
  - Requires suitable representation for symbolic result
  - Depends on strength of symbolic reasoning
  - Must analyze recursion patterns
  - Worst cases:
    - $trans_f = \top$ (i.e., as imprecise as intra-procedural analysis)
    - $trans_f$ is symbolic expression as complex as f itself

Heap Analysis

Limitations

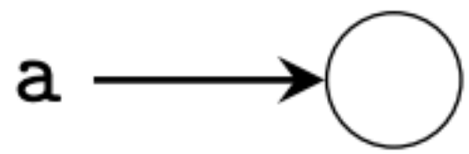
```
a = new A();
b = a;
c = new A();
a.x = 17;
c.x = 23;
b.x = 42;
print(a.x);
print(c.x);
```

- Program Analysis 1:
  - a.x and b.x and c.x are different things
  - We print 17 and 23
  - Unsound analysis!
- Program Analysis 2:
  - a and c and b might point to the same object
  - We print one of {17, 23, 42}, twice
  - Imprecise analysis
- Need to know:
  - Which variables can be aliases of each other?

Heap Graph

Example

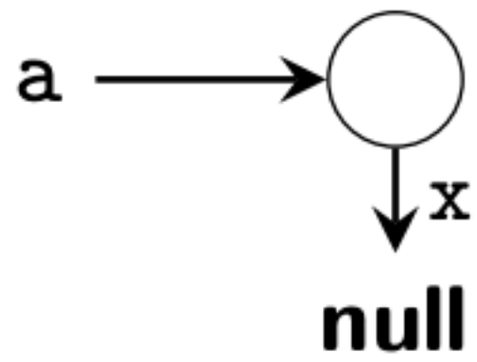
```
a := new();    // ⇐
a.x := null;
b := a;
b.x := new();
a.x.y := 1;
c := new();
c.x := new();
c.x.x := a;
c := a.x;
// A
```



Heap Graph

Example

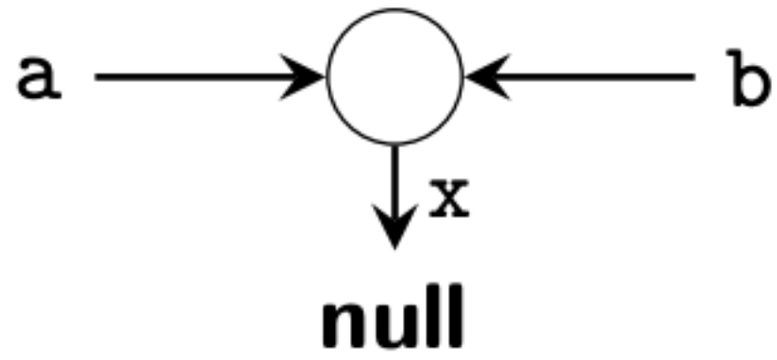
```
a := new();
a.x := null;  // ⇐
b := a;
b.x := new();
a.x.y := 1;
c := new();
c.x := new();
c.x.x := a;
c := a.x;
// A
```



Heap Graph

Example

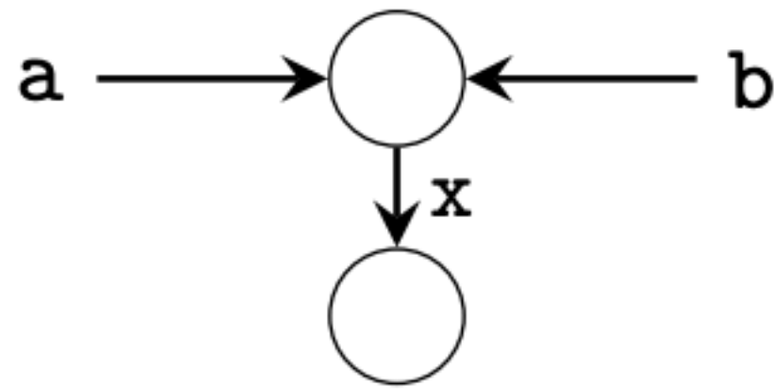
```
a := new();
a.x := null;
b := a;                    // ⇐
b.x := new();
a.x.y := 1;
c := new();
c.x := new();
c.x.x := a;
c := a.x;
// A
```



Heap Graph

Example

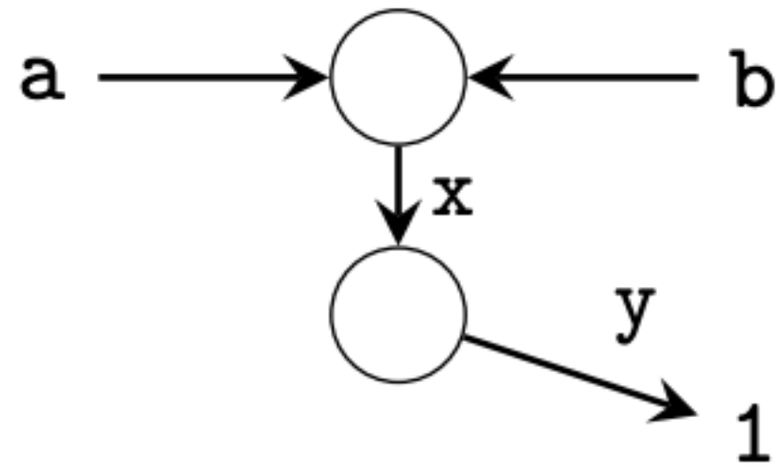
```
a := new();
a.x := null;
b := a;
b.x := new(); // ⇐
a.x.y := 1;
c := new();
c.x := new();
c.x.x := a;
c := a.x;
// A
```



Heap Graph

Example

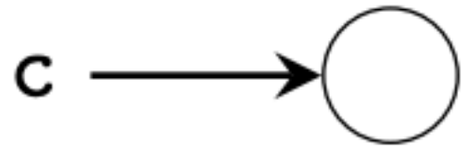
```
a := new();
a.x := null;
b := a;
b.x := new();
a.x.y := 1;    // ⇐
c := new();
c.x := new();
c.x.x := a;
c := a.x;
// A
```

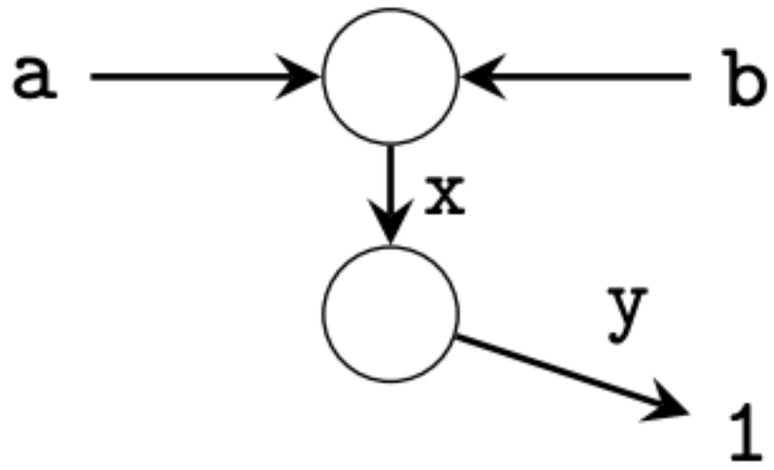


Heap Graph

Example

```
a := new();
a.x := null;
b := a;
b.x := new();
a.x.y := 1;
c := new();    // ⇐
c.x := new();
c.x.x := a;
c := a.x;
// A
```

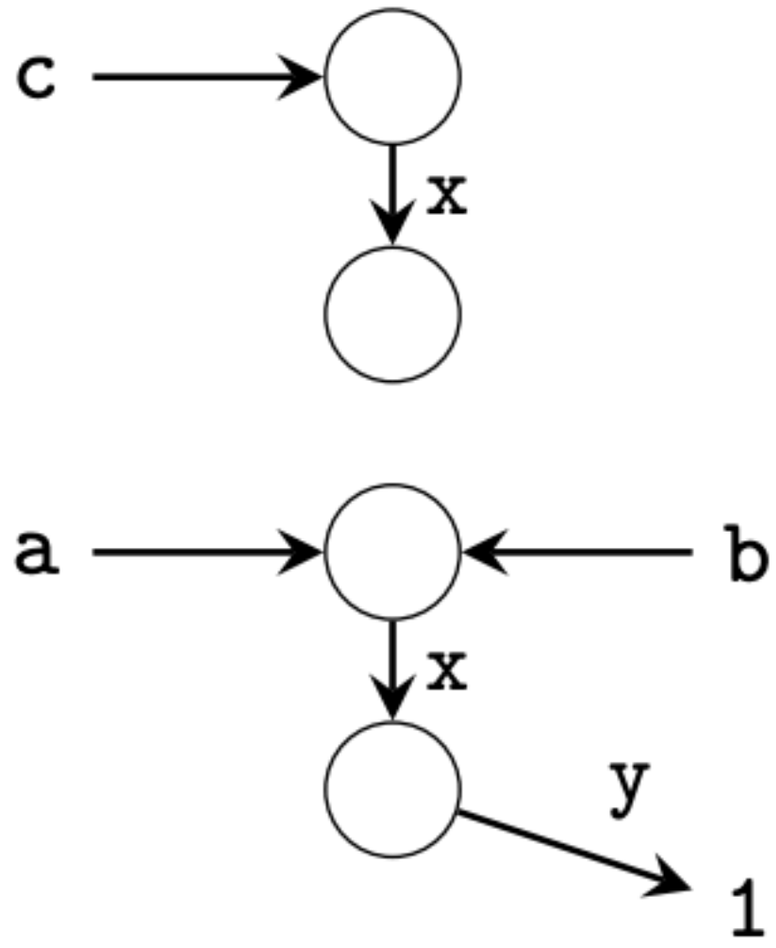




Heap Graph

Example

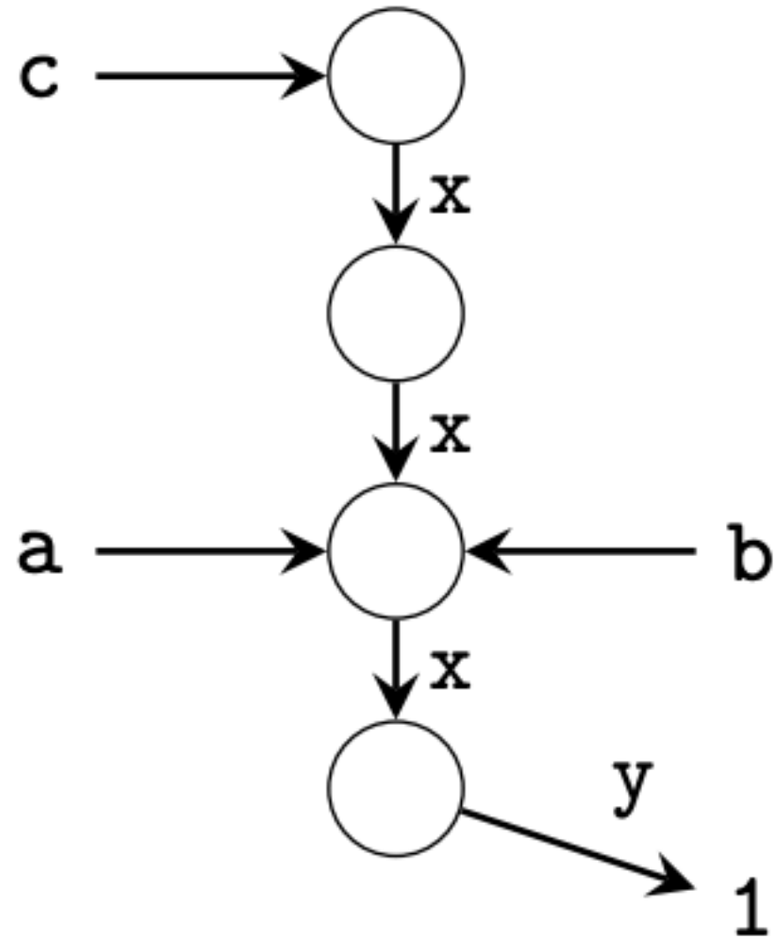
```
a := new();
a.x := null;
b := a;
b.x := new();
a.x.y := 1;
c := new();
c.x := new(); // ⇐
c.x.x := a;
c := a.x;
// A
```



Heap Graph

Example

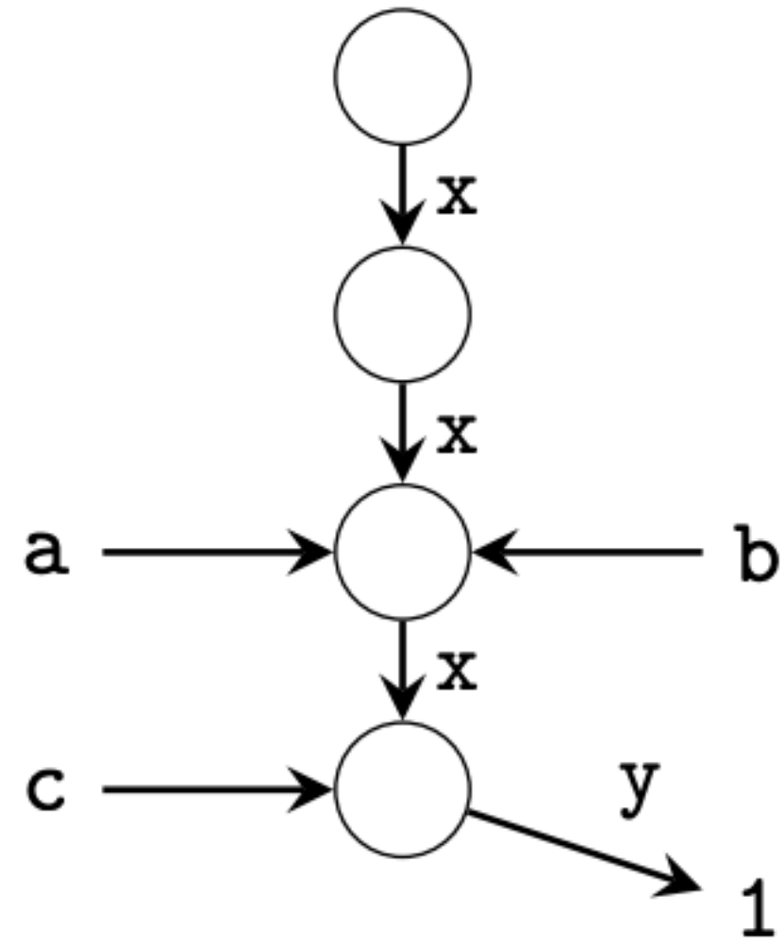
```
a := new();
a.x := null;
b := a;
b.x := new();
a.x.y := 1;
c := new();
c.x := new();
c.x.x := a;    // ⇐
c := a.x;
// A
```



Heap Graph

Example

```
a := new();
a.x := null;
b := a;
b.x := new();
a.x.y := 1;
c := new();
c.x := new();
c.x.x := a;
c := a.x;      // ⇐
// A
```



Example

```
a := new();
a.x := null;
b := a;
b.x := new();
a.x.y := 1;
c := new();
c.x := new();
c.x.x := a;
c := a.x;
// A
```

Aliases at // A:

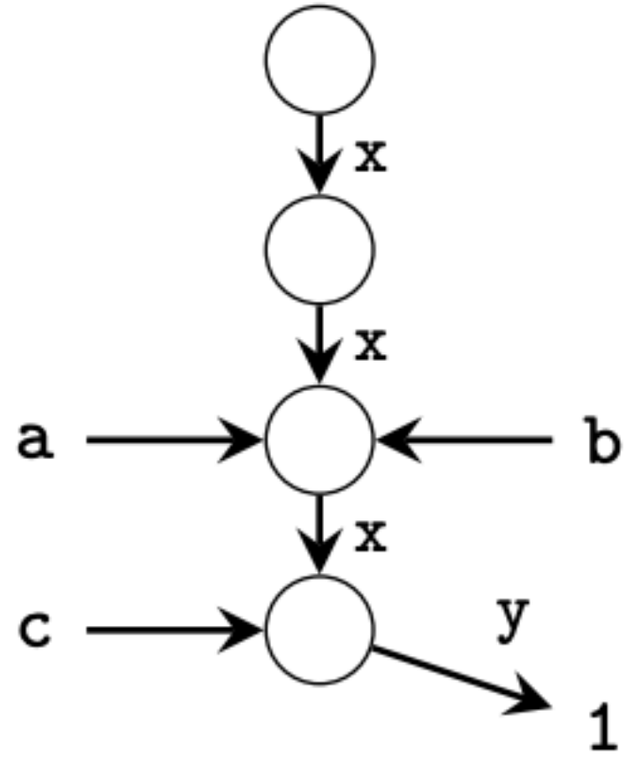
- a and b represent the same object

$\Rightarrow$ a and b are *aliased*

$$a \overset{alias}{=\!=} b$$

$\Rightarrow$ a.x and b.x are *aliased*

- c and a.x and b.x are *aliased*



Pointer Operations

Referencing	Dereferencing	Aliasing
C	C	C
`my_t *p = &var;` `p = malloc(8);`	`int x = *ptr;` `x = ptr2->field;`	`my_t *pa;` `pa = pb;`
Java	Java	Java
`A a = new A();`	`int x = a.f;`	`A b = a;`

Points-to Information

- Variable pairs that refer to the same memory location
  - <*a, b>, <*c, b>, <*a, *c>
  - *a and b alias, same with *c and b
- Points-to pairs:
  - <a → b>, <c → b>
  - a points to b, and c points to b (hence *a and *c are alias)
- Alias sets:
  - {*a, b, *c}
  - They all point to the same memory location

Locations

- Static locations
  - Uniquely identified by name
- Stack-dynamic locations (Local vars, parameters)
  - Named
  - May never exist (function not called)
  - May exist more than once (recursion)
- Heap-dynamic locations
  - Not named
  - May exist arbitrarily often

Heap/Stack Locations: Design Space

- Don't distinguish, or
- Distinguish by:
  - Type
  - Name
  - Calling context (Context-sensitive analysis)
  - Control flow (Flow-sensitive analysis)
  - Containing heap objects (Object sensitivity)
  - Referencing field name in objects (field sensitivity)
  - Array index (array sensitivity)
  - Pointer arithmetic
  - Literature has explored many options

Steensgaard's Points-To Analysis

- Steensgaard, B. (1996, January). Points-to analysis in almost linear time. In Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (pp. 32-41).

- Fast: $O(n\alpha(n,n))$ over variables in program
- Developed to deal with large code bases at AT&T
- Sacrifices Precision
- Equality-based
- Basis for many more precise algorithms

Steensgaard's Points-To Analysis

- C pointer semantics:
  - &a: Address of a
  - *a: Object pointed to by a
  - Converse operators: *(&a) = a
- Steensgaards analysis considers four cases:

	C	Java	
Referencing	`a = &b`	`a = new A()`	$\ell_b \in$ pts(a)
Aliasing	`a = b`	`a = b`	pts(a) = pts(b)
Dereferencing read	`a = *b`	`a = b.f`	for each $\ell \in$ pts(b) → pts(a) = pts(ℓ)
Dereferencing write	`*a = b`	`a.f = b`	for each $\ell \in$ pts(a) → pts(b) = pts(ℓ)

Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:

Steensgaard:

Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

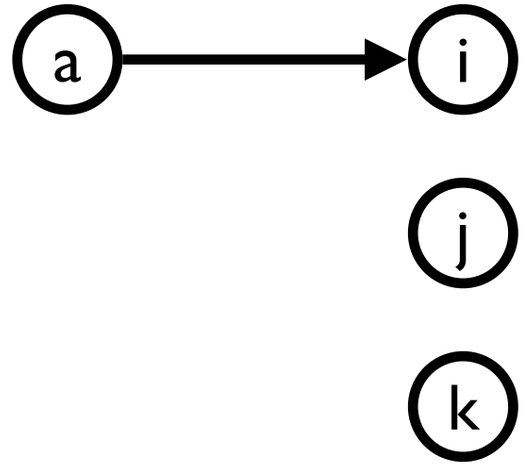
Actual:

Steensgaard:

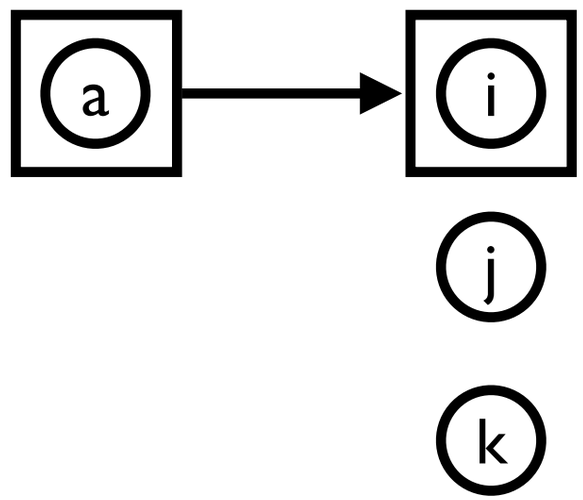
Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



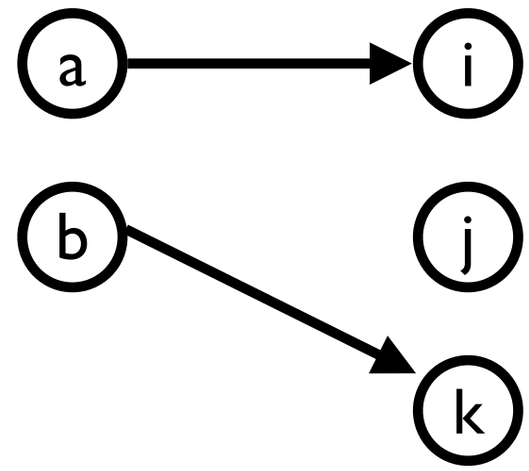
Steensgaard:



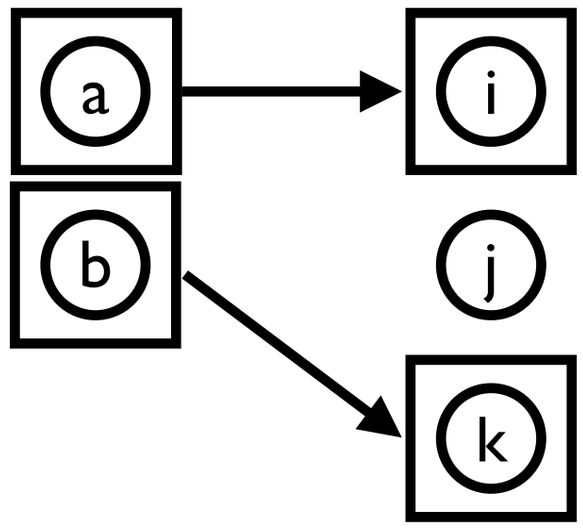
Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



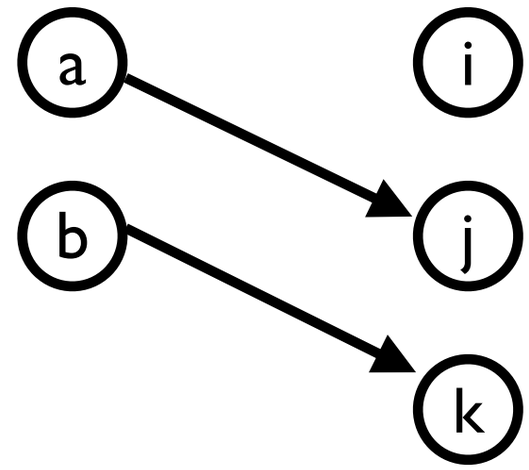
Steensgaard:



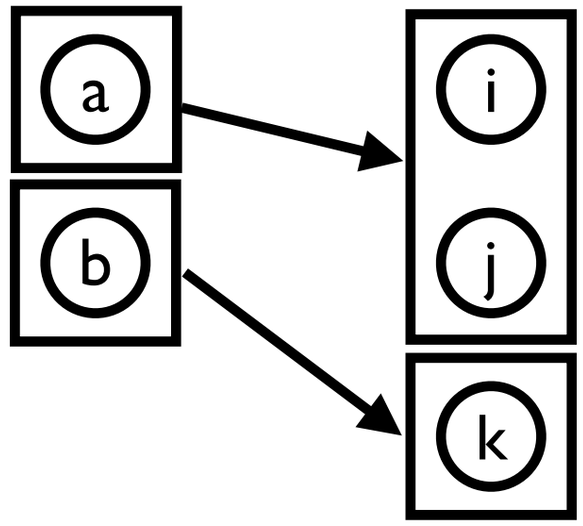
Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



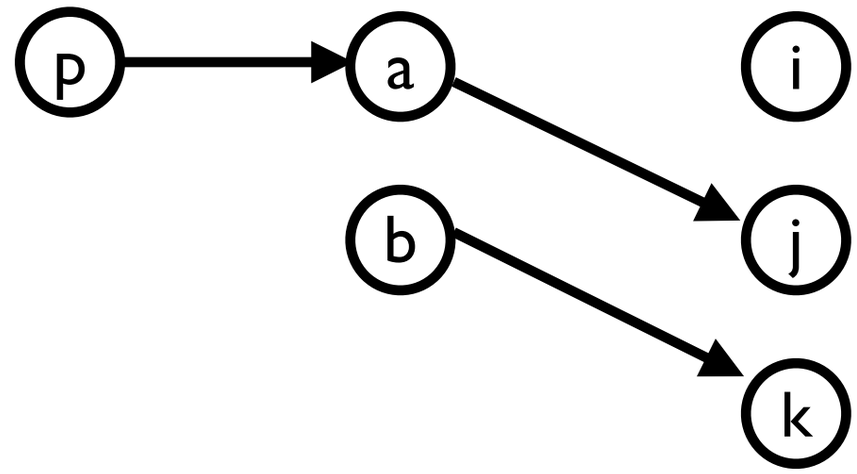
Steensgaard:



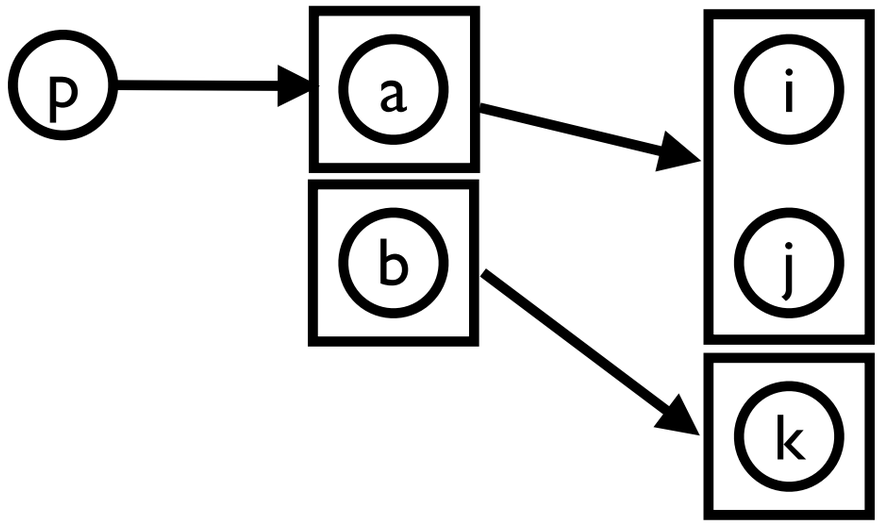
Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



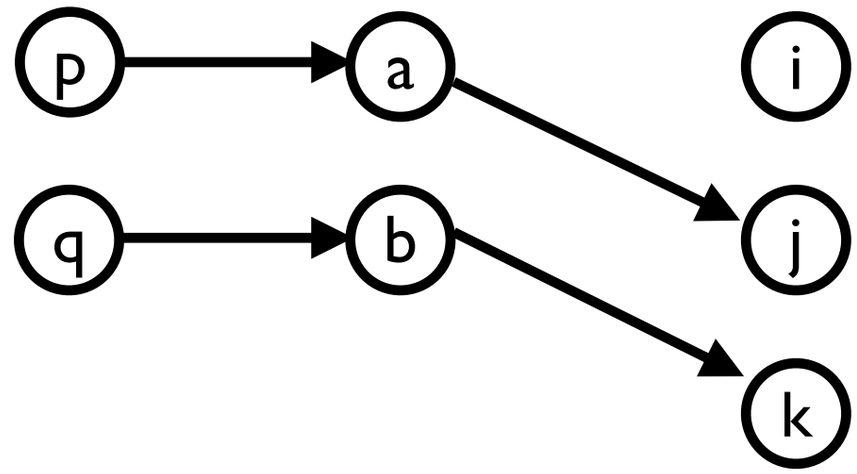
Steensgaard:



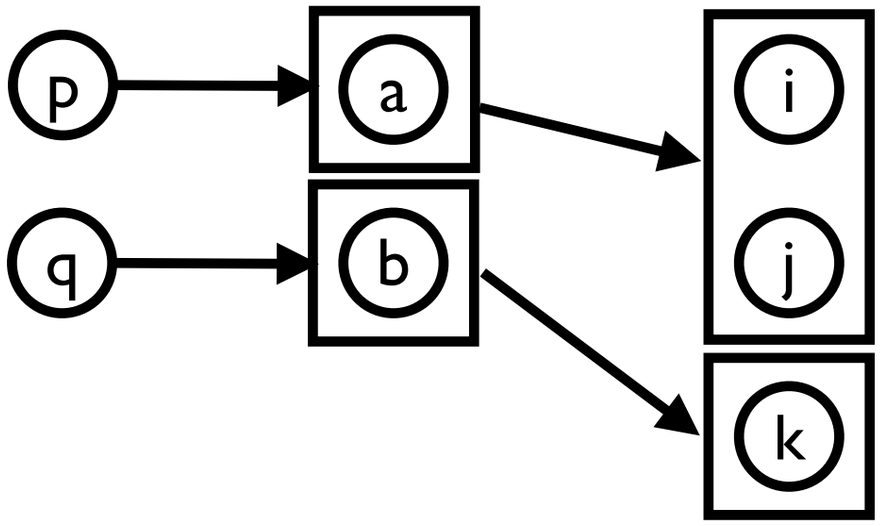
Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



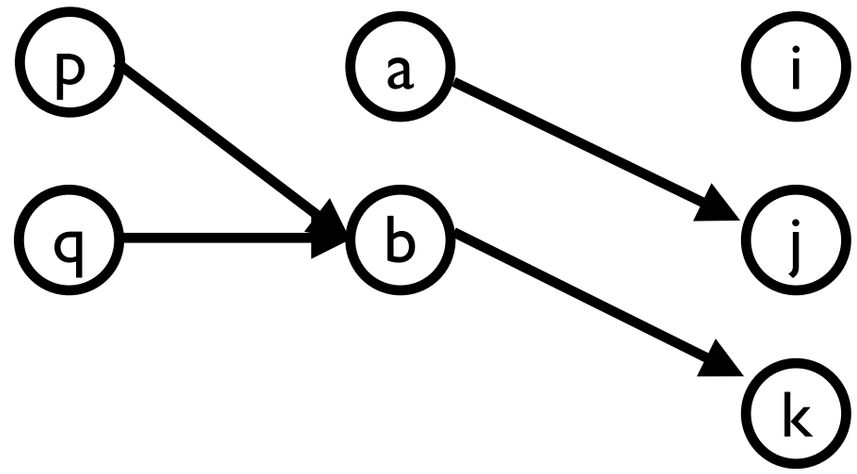
Steensgaard:



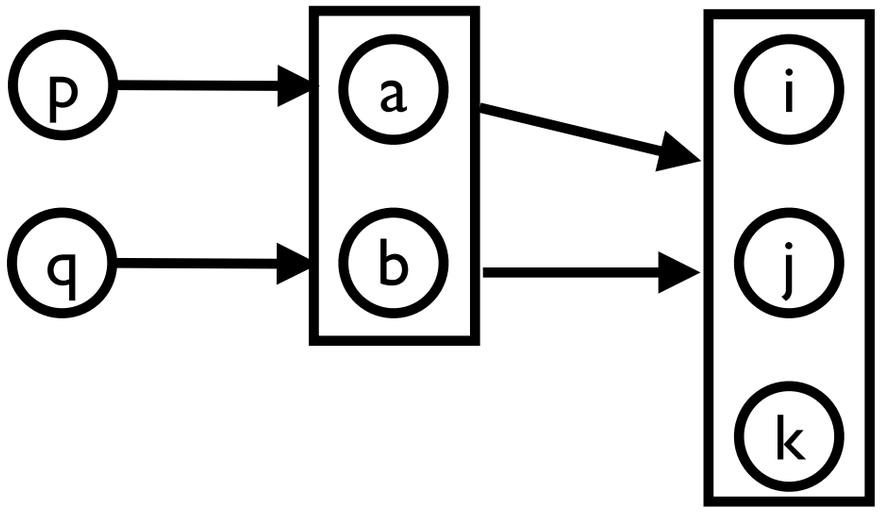
Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



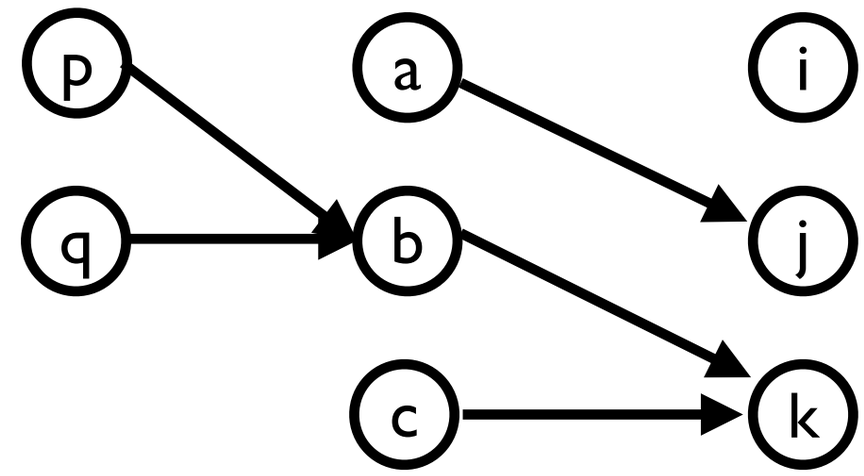
Steensgaard:



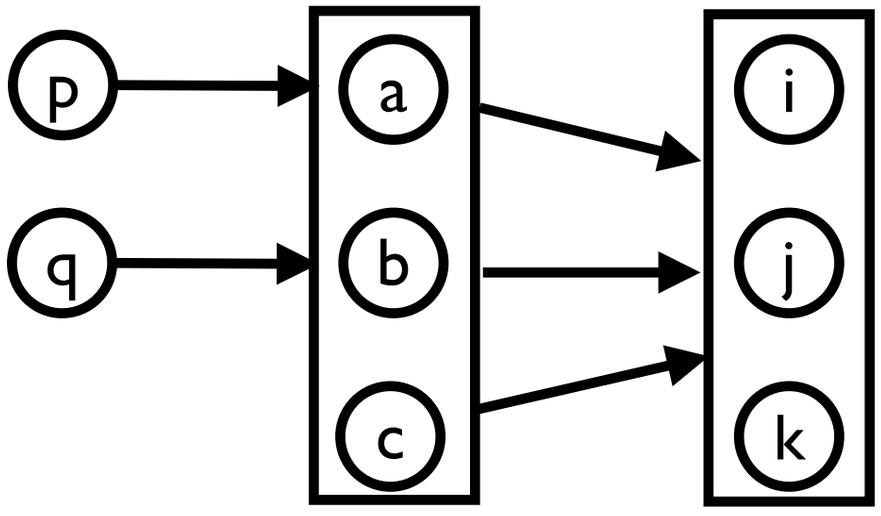
Steensgaard's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



Steensgaard:



Steensgaard's Points-To Analysis

- Points-to sets pts(v) serve as abstraction over addresses that v can point to
- Steensgaard's points-to-analysis:
  - Insensitive to flow, context, fields, …
- Efficiently implemented using Union-Find data structure
- Highly efficient
- Imprecise

Andersen's Points-To Analysis

- Andersen, L. O. (1994). Program analysis and specialization for the C programming language (Doctoral dissertation, University of Copenhagen).

- Asymptotic performance is $O(n^3)$

- More precise than Steensgaard's analysis
  - Still not terribly precise

- Subset-based (a.k.a. inclusion-based)

- Basis for many algorithms with performance improvements

Andersen's Points-To Analysis

- Collect constraints, resolve as needed
- For each statement in program, we record:
  - If Referencing (a = &b):
    $\{\ell_b\} \subseteq$ pts(a)
  - If Aliasing (a = b):
    pts(b) ⊆ pts(a)
  - If Dereferencing read (a = *b):
    pts(*b) ⊆ pts(a)
  - If Dereferencing write (*a = b):
    pts(b) ⊆ pts(*a)

Andersen's Points-To Analysis

Actual:

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Andersen:

Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

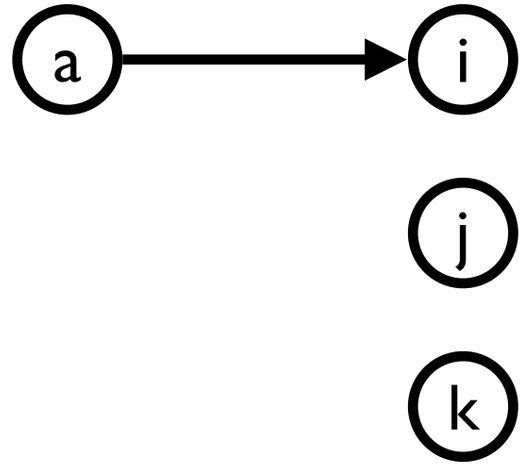
Actual:

Andersen:

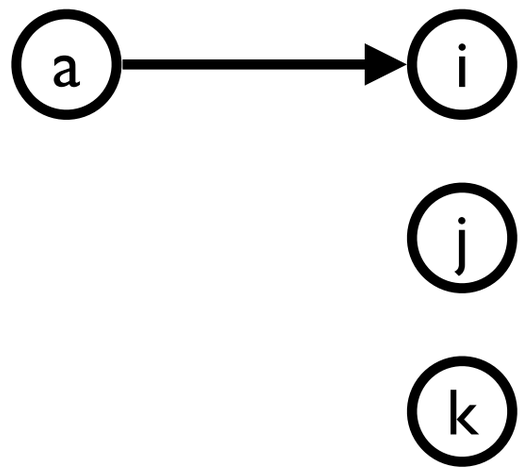
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



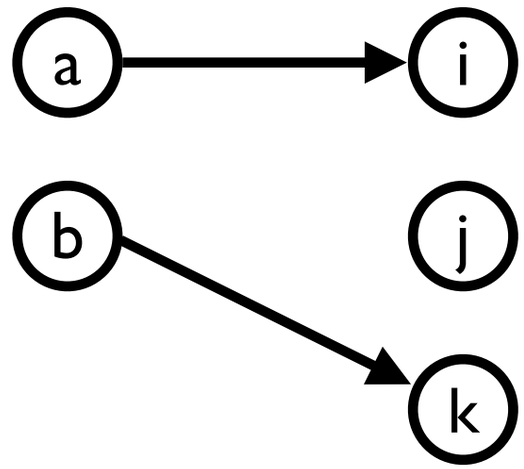
Andersen:



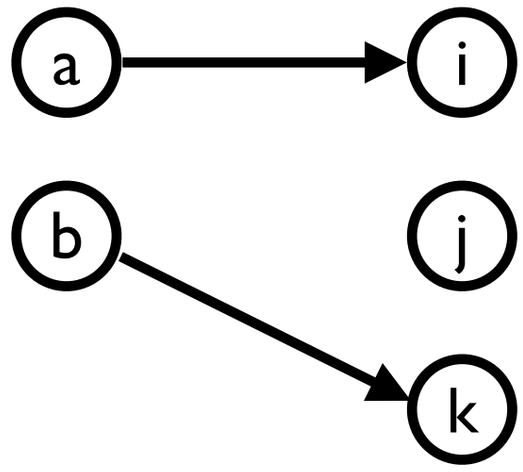
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



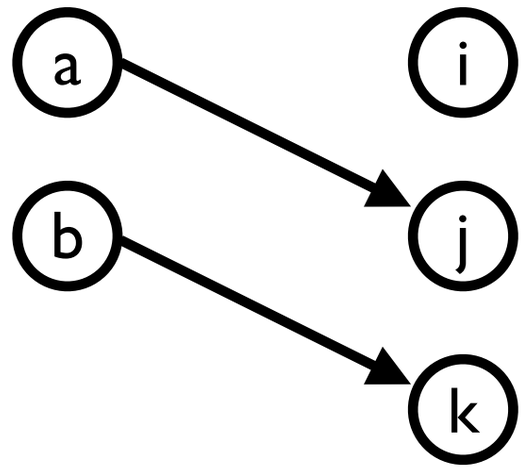
Andersen:



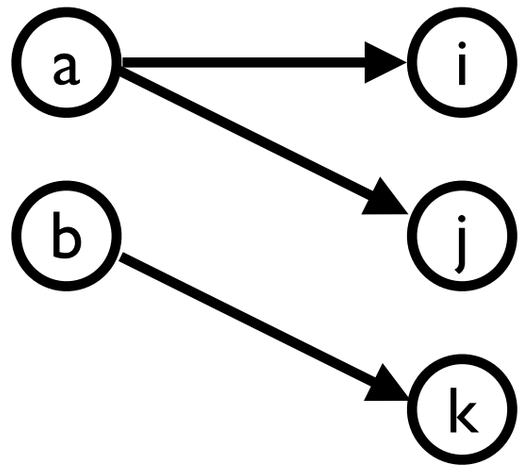
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



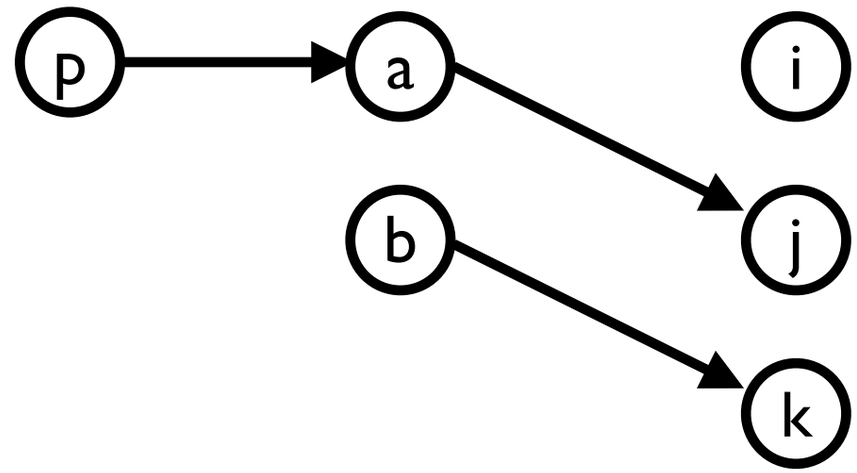
Andersen:



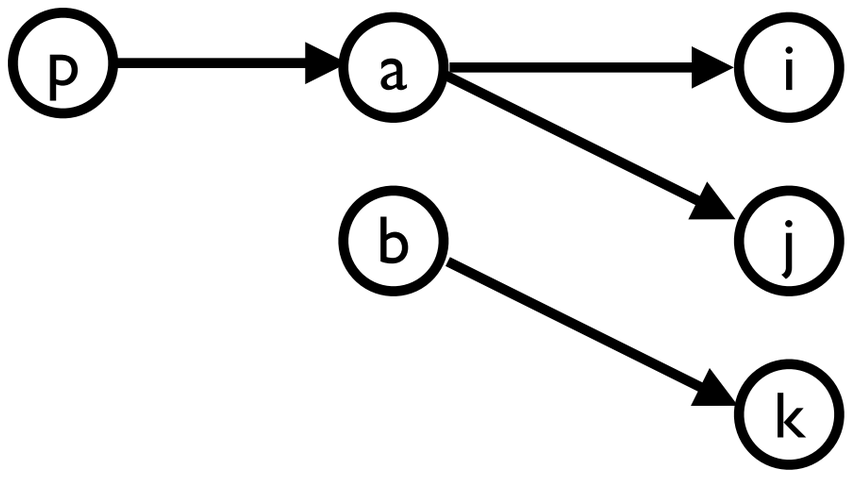
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



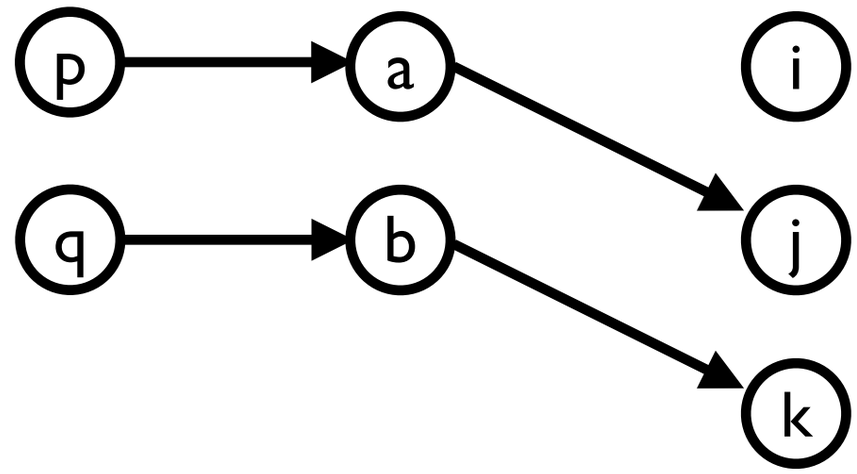
Andersen:



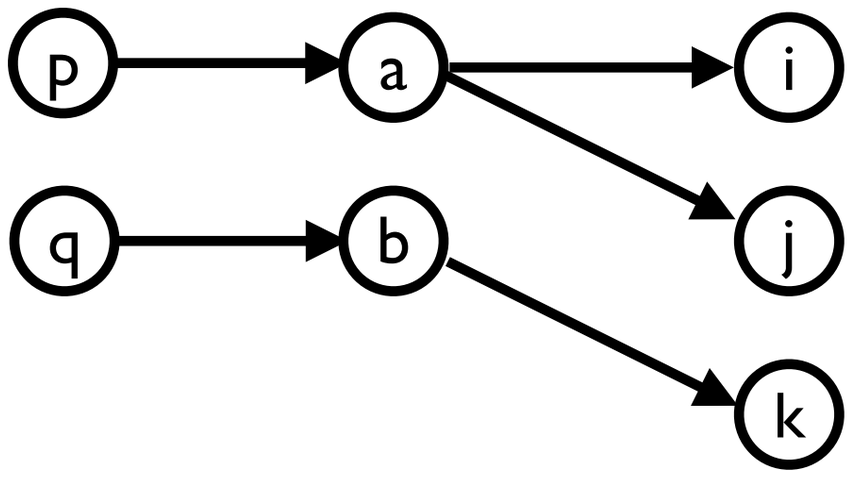
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



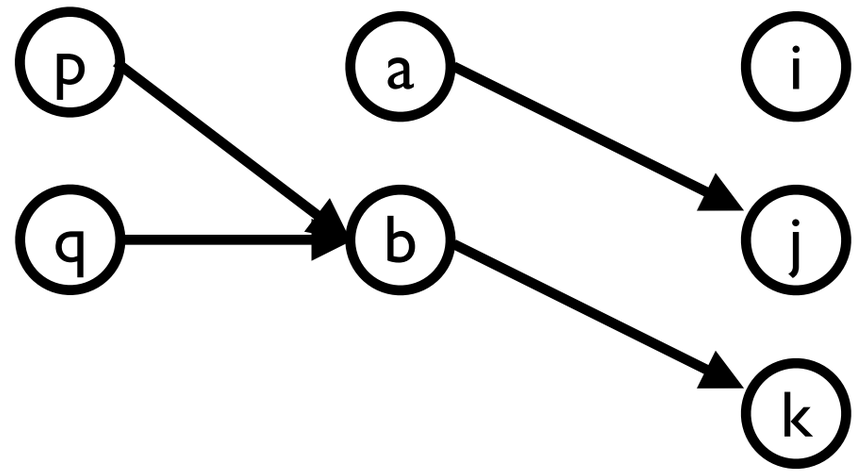
Andersen:



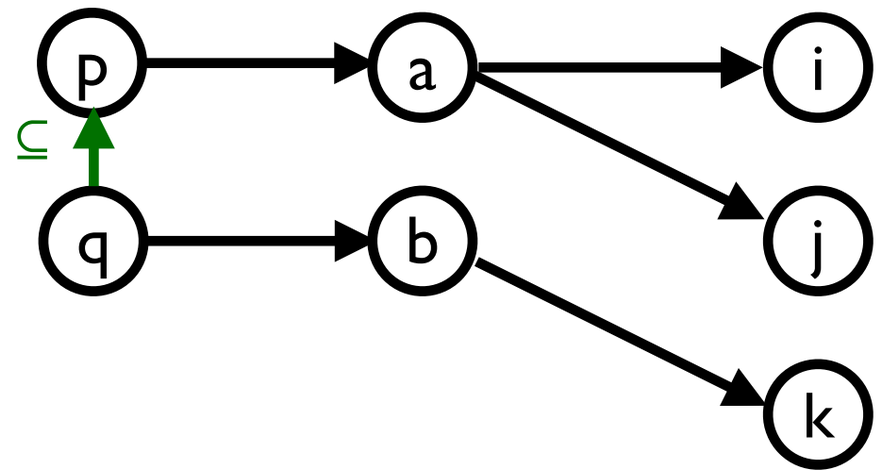
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



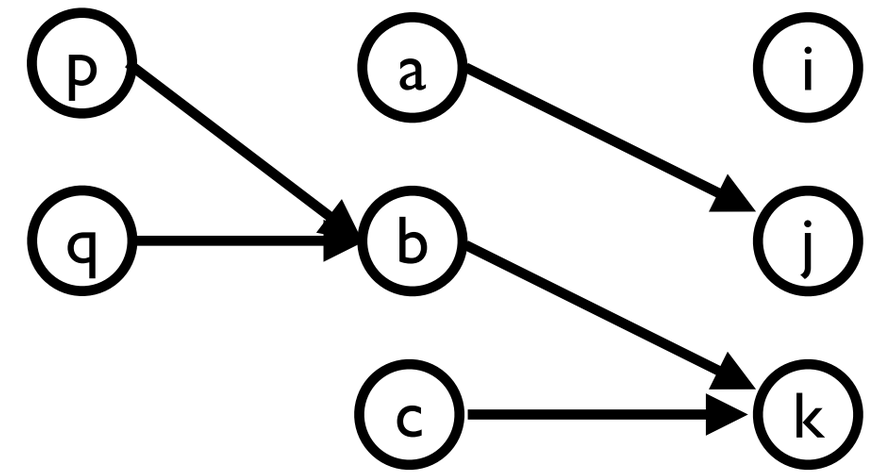
Andersen:



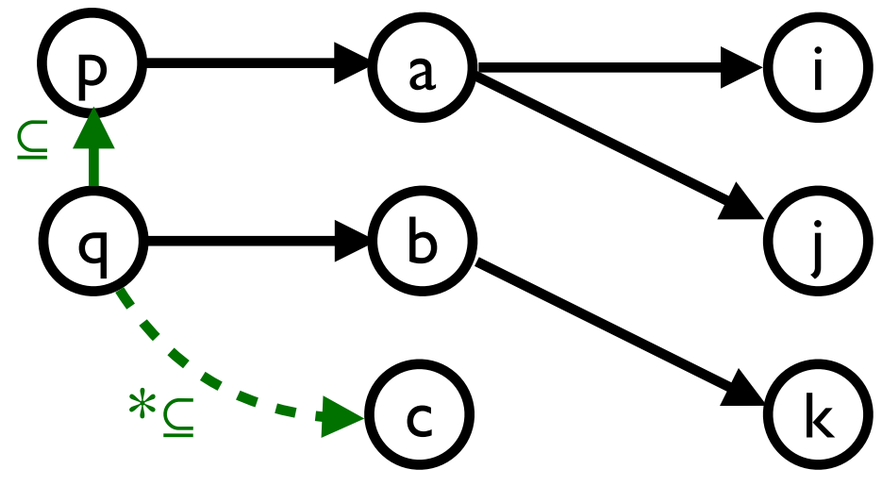
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



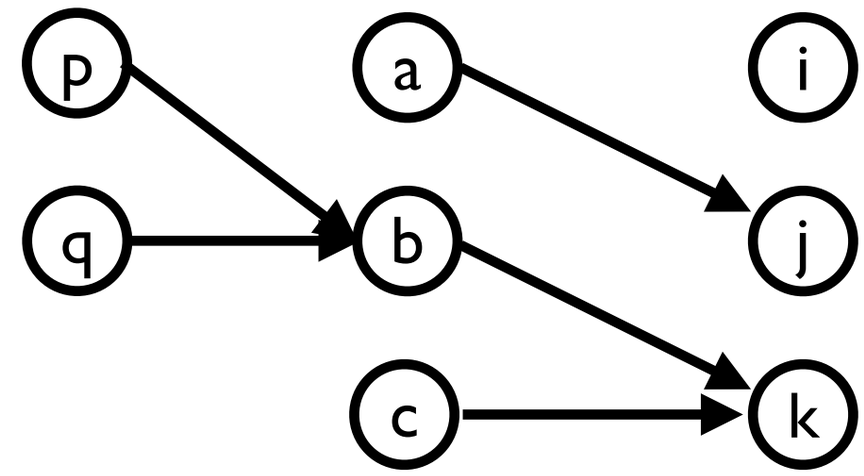
Andersen:



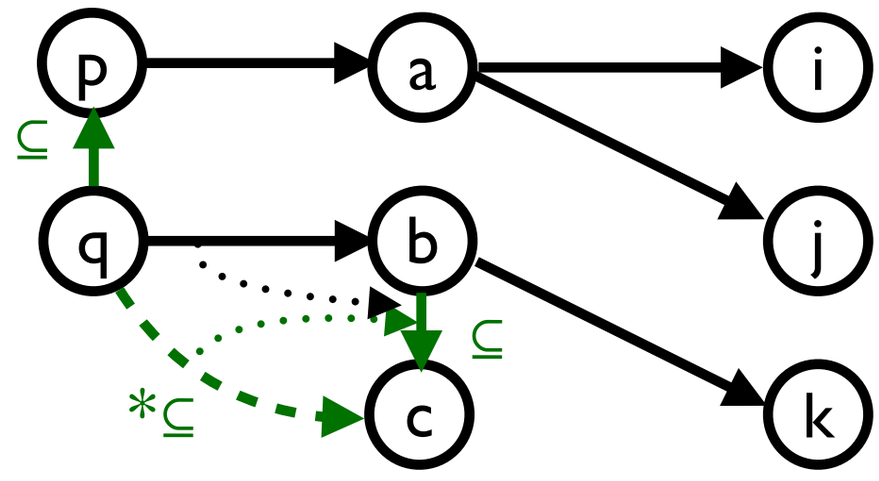
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



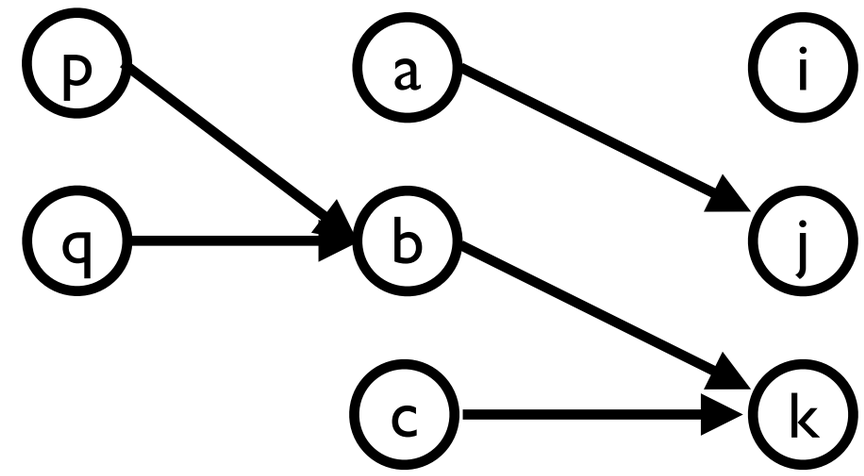
Andersen:



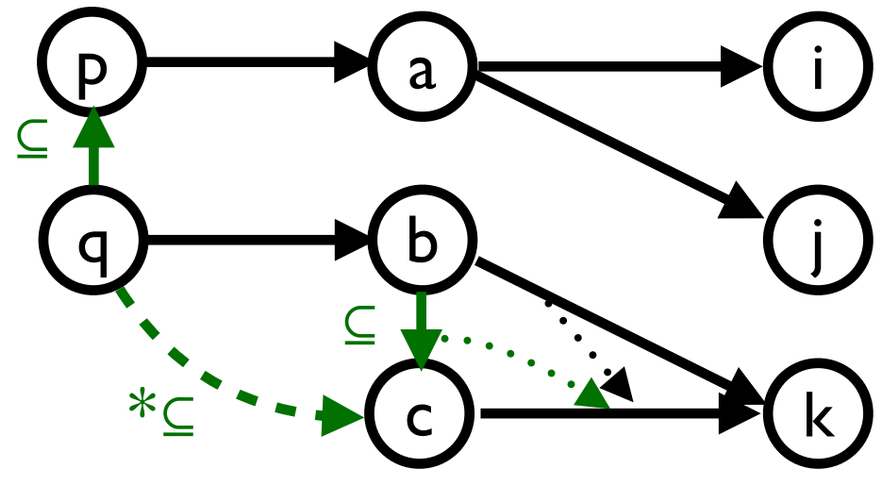
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



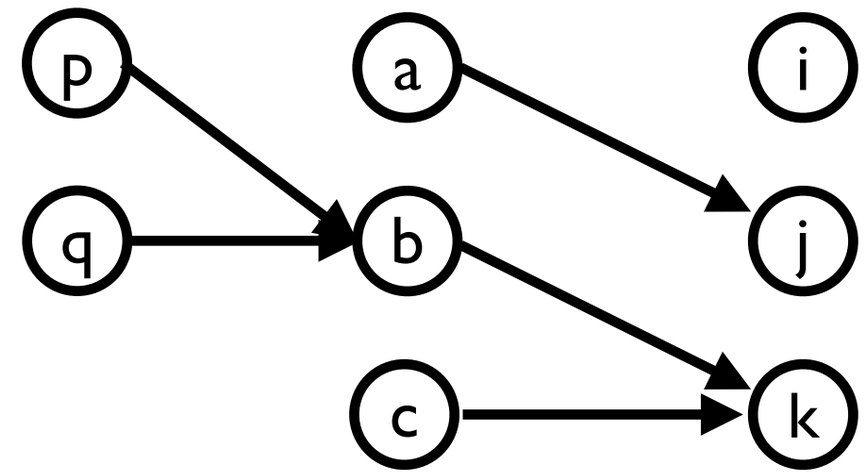
Andersen:



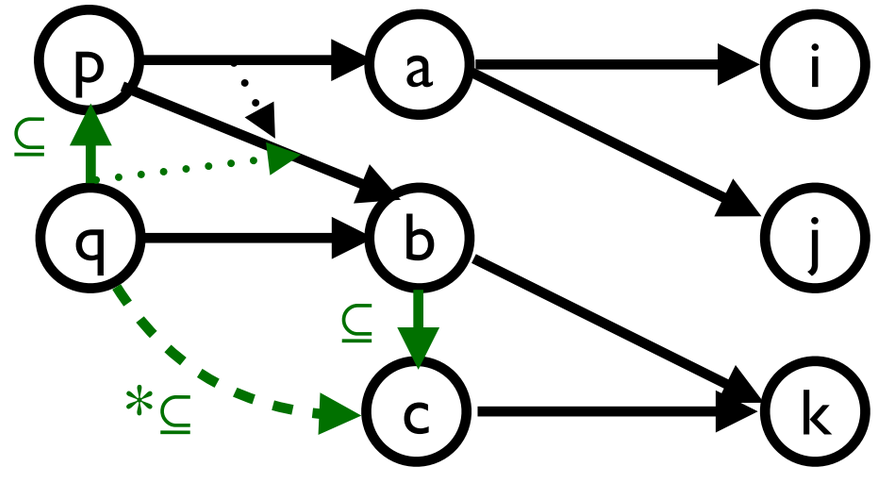
Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



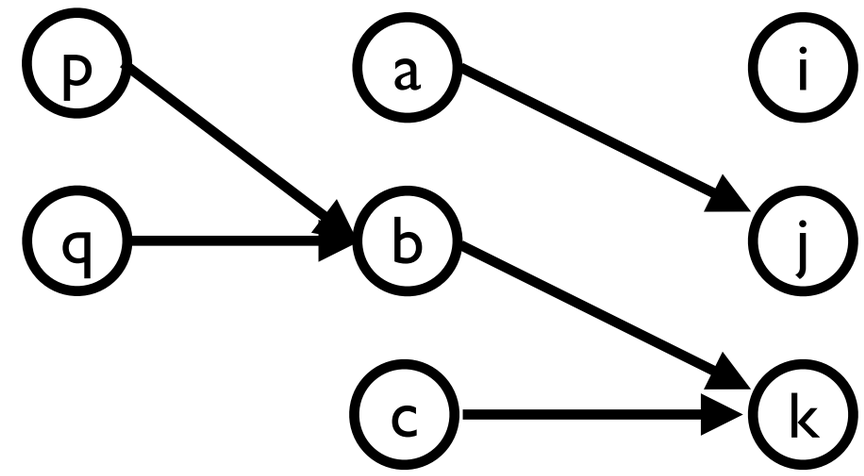
Andersen:



Andersen's Points-To Analysis

```
int i, j, k;
int *a = &i;
int *b = &k;
a = &j;
int **p = &a;
int **q = &b;
p = q;
int *c = *q;
```

Actual:



Andersen: